Prototipo de sistema de traducción de Lengua de Señas Argentina con procesamiento en tiempo cuasi real

Maite Nigro Francisco Veron

Director: Esp. Ing. Gonzalo Avalos Ribas

Co-director: Ing. Bruno Constanzo

Universidad Nacional de Mar del Plata
Facultad de Ingeniería
Departamento de Ingeniería en Informática

Marzo 2026

Agradecimientos

De Maite: Para la Flaqui.
De Francisco: Para el Diego y Baco.
A Miriam Rolls, por su orientación sobre la LSA y la validación del vocabulario del conjunto de datos.

Índice general

1. Introducción **1**
- 1.1. Contexto . . . 1
- 1.2. Motivación . . . 1
- 1.3. Problemática . . . 1
- 1.4. Alcance y limitaciones . . . 2
  - 1.4.1. Delimitación del vocabulario y asesoramiento lingüístico . . . 2
  - 1.4.2. Límites operativos y comportamiento ante entradas no contempladas 3
  - 1.4.3. Infraestructura de despliegue . . . 4

2. Objetivos **5**
- 2.1. Objetivo general . . . 5
- 2.2. Objetivos específicos . . . 5

3. Marco teórico, estado del arte y metodología **7**
- 3.1. Marco teórico . . . 7
  - 3.1.1. Componentes articulatorios y estructura fonológica de la LSA . . . 7
  - 3.1.2. Estructura lingüística de la LSA y orden manual del prototipo . . . 7
  - 3.1.3. Sistemas de tiempo real y tiempo pseudo-real . . . 8
  - 3.1.4. Visión computacional y reconocimiento basado en esqueletos . . . 8
  - 3.1.5. Modelado de secuencias temporales y atención . . . 9
  - 3.1.6. Modelos de lenguaje pequeños y adaptación *LoRA* . . . 10
  - 3.1.7. Estándares de captura web y arquitecturas de extensión web . . . 11
- 3.2. Estado del arte . . . 11
- 3.3. Metodología de desarrollo . . . 12
  - 3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial . . 14
- 3.4. Herramientas y tecnologías empleadas . . . 15

4. Análisis de requerimientos **16**
- 4.1. Requerimientos funcionales . . . 16
- 4.2. Requerimientos no funcionales . . . 17
- 4.3. Restricciones . . . 17
  - 4.3.1. Especificación del entorno de ejecución cliente . . . 18
- 4.4. Casos de uso . . . 19
  - 4.4.1. Casos de uso esenciales en formato breve . . . 19
  - 4.4.2. Casos de uso expandidos . . . 19

5. Diseño del Sistema **25**
5.1. Arquitectura del sistema . . . 25
5.1.1. Justificación de la partición y modelo de privacidad . . . 25
5.1.2. Flujo integral de datos entre componentes . . . 25
5.2. Diseño de contratos de API REST . . . 25
5.2.1. API local del motor ILSA . . . 25
5.2.2. API del servicio semántico remoto . . . 26
5.2.3. API del servicio de archivos del avatar . . . 26
5.3. Diseño de la representación gestual inversa . . . 26
5.3.1. Procesamiento geométrico de coordenadas y segmentación temporal 28
5.3.2. Preparación, acondicionamiento y ensamblado cinemático . . . 29
5.3.3. Estrategia de contingencia léxica . . . 31
5.4. Diseño de la extensión de videollamada . . . 31
5.5. Patrones de diseño de software aplicados . . . 31
5.6. Diseño conceptual de la arquitectura escalable de producción en la nube . . 32
5.6.1. Ingreso . . . 32
5.6.2. Traductor . . . 33
5.6.3. Archivos de seña . . . 33
5.6.4. Monitoreo . . . 33
5.6.5. Qué queda fuera de las pruebas . . . 33

6. Implementación y Desarrollo **34**
6.1. Estructura del código . . . 34
6.1.1. Repositorio del clasificador . . . 34
6.1.2. Cliente de la videollamada . . . 34
6.1.3. Servicio semántico en la nube . . . 34
6.1.4. Entrenamiento del canal directo . . . 35
6.1.5. Entrenamiento del canal inverso . . . 35
6.1.6. Servicio de poses del avatar . . . 35
6.2. Implementación del subsistema de reconocimiento gestual . . . 35
6.2.1. Preprocesamiento de datos y pipeline de extracción . . . 35
6.2.2. Arquitectura neuronal TinySkeletonClassifier . . . 36
6.3. Implementación del subsistema de traducción semántico bidireccional . . . 36
6.3.1. Selección y especificación de la suite de modelos de lenguaje . . . 36
6.3.2. Régimen de entrenamiento y ajuste LoRA . . . 36
6.3.3. Canal directo (LSA → Español) . . . 38
6.3.4. Canal inverso (Español → LSA) . . . 39
6.3.5. Cuantización GGUF Q4_K_M y estimación de memoria . . . 39
6.4. Integración en Google Meet y arquitectura de la extensión . . . 41
6.4.1. Secuencia completa de inferencia y comunicación multidireccional . 41
6.4.2. Motor local ILSA (FastAPI) . . . 44
6.5. Implementación del avatar . . . 44
6.5.1. Vía descartada: retarget hacia un avatar VRM . . . 44
6.5.2. Decisiones técnicas de la reproducción . . . 45
6.6. Instancia de validación del prototipo mediante microservicios en la nube . . 46

7. Pruebas, Resultados y Discusión **47**
7.1. Corpus y metodología de adquisición de datos de LSA 47
7.1.1. Definición y validación del vocabulario de 100 clases 47
7.1.2. Protocolo de grabación y curación del dataset propio 47
7.2. Evaluación empírica del clasificador gestual (TinySkeletonClassifier) 48
7.2.1. Protocolo de evaluación en cámara real 48
7.2.2. Comparativa entre métricas fuera de línea y rendimiento en cámara 49
7.2.3. Impacto de la resolución temporal: 16 frente a 32 fotogramas 49
7.2.4. Búsqueda automatizada con Optuna y análisis de sobreajuste . . . 51
7.2.5. Diagnóstico de confusiones mediante feature engineering 52
7.2.6. Impacto cuantitativo de la corrección de relación de aspecto 52
7.2.7. Desempeño final del clasificador en tiempo real 53
7.3. Evaluación de los módulos de traducción semántica 56
7.3.1. Estrategia de generación asistida del dataset 56
7.3.2. Corrida histórica previa a la compresión de tokens (Canal Directo) . 57
7.3.3. Resultados finales y reevaluación semántica 57
7.3.4. Análisis cualitativo de errores en la inferencia 58
7.3.5. Análisis de asimetría direccional 59
7.3.6. Compromiso entre latencia y capacidad lingüística 59
7.4. Evaluación del sistema integrado y rendimiento en tiempo real 59
7.4.1. Validación funcional de extremo a extremo 59
7.4.2. Análisis de latencias totales por etapa del pipeline 61
7.4.3. Consumo de recursos de hardware: máquina de desarrollo frente a equipo objetivo 62
7.5. Discusión técnica y lecciones de ingeniería aprendidas 62

8. Conclusiones y trabajo futuro **64**
8.1. Conclusiones 64
8.1.1. Cumplimiento de objetivos del proyecto 64
8.1.2. Aportes técnicos y de accesibilidad 64
8.1.3. Impacto social en el marco de la Ley 27.710 64
8.2. Trabajo futuro 64
8.2.1. Implementación física del clúster semántico escalable 64
8.2.2. Expansión del corpus multiseñante y mitigación de variabilidad . . 64
8.2.3. Incorporación de rasgos no manuales (expresiones faciales) 64
8.2.4. Generalización a nuevas plataformas de comunicación virtual 64

A. Vocabulario del clasificador **68**

C. Arquitectura escalable de producción en la nube **69**

Resumen

En la actualidad, la interacción virtual entre personas sordas y oyentes se limita casi exclusivamente a la comunicación escrita o a la transcripción unilateral de voz a texto, lo que genera una marcada asimetría en la comunicación expresiva durante videollamadas. Este Trabajo Final de Grado desarrolla un prototipo de traducción bidireccional entre la Lengua de Señas Argentina (LSA) y el español oral estructurado bajo una arquitectura híbrida de computación en el borde y servicios en la nube con procesamiento en tiempo pseudo-real.

El sistema articula dos canales comunicacionales complementarios. En el sentido señante-oyente, la computadora del usuario captura localmente el flujo de video desde la cámara web, extrae coordenadas articulares tridimensionales mediante visión computacional y clasifica un vocabulario de 100 señas mediante una red neuronal ligera (Conv1D + Tiny Transformer con *attention pooling*). Con el fin de evitar la saturación de memoria y cómputo que provocaría ejecutar modelos de lenguaje generativos en dispositivos convencionales, las glosas clasificadas son enviadas hacia un backend semántico en la nube que transforma la secuencia en oraciones fluidas en español rioplatense, las cuales retornan al cliente para su proyección en subtítulos o síntesis de voz. En sentido inverso (oyente-señante), el sistema procesa la voz y/o texto del interlocutor oyente mediante un módulo semántico inverso en la nube que estructura las glosas, enviándolas a un avatar en el cliente que ejecuta las animaciones correspondientes o recurre al deletreo dactilológico para términos fuera del léxico.

El módulo clasificador gestual local alcanza un 98,28 % de exactitud en validación fuera de línea y un 77,3 % de acierto en cámara real. Por su parte, los modelos de lenguaje evaluados (Qwen2.5, Llama 3.2 y SmolLM adaptados mediante LoRA y cuantizados a GGUF Q4_K_M) superan el 94 % de exactitud y alcanzan latencias de inferencia de entre 200 y 450 ms. Como mecanismo de integración para videollamadas, el sistema se acopla a Google Meet mediante una extensión de navegador conectada al motor local ILSA, y asegura la privacidad del usuario al no transmitir video crudo fuera del dispositivo cliente. Finalmente, se deja documentada la propuesta de diseño de una arquitectura en la nube escalable para soportar la demanda concurrente en entornos de producción.

Palabras clave:

Lengua de Señas Argentina, reconocimiento de gestos, arquitectura híbrida cliente-nube, traducción semántica bidireccional, avatar 3D, accesibilidad.

Capítulo 1

Introducción

1.1. Contexto

Tradicionalmente, la sociedad abordó a la comunidad sorda desde una perspectiva clínica centrada en la rehabilitación y la corrección auditiva. Ese enfoque omite que la comunidad sorda se identifica, ante todo, como una minoría lingüística con cultura y gramática propias, independientes del español oral. En la Argentina, la Lengua de Señas Argentina (LSA) cuenta con reconocimiento legal formal a través de la Ley 27.710 [1], la cual garantiza su preservación y el derecho de las personas sordas a comunicarse en su lengua natural en todos los ámbitos sociales.

En el contexto digital contemporáneo, las plataformas de videoconferencia se han convertido en herramientas centrales para el ámbito laboral, educativo y de la vida cotidiana. Sin embargo, los *software* de uso masivo presentan escasa integración de herramientas automatizadas que admitan la LSA como un flujo comunicacional bidireccional en tiempo pseudo-real. Los recursos tecnológicos existentes suelen restringirse a diccionarios estáticos o repositorios de video, carentes de capacidad de interpretación contextual e interactiva durante una llamada.

1.2. Motivación

Este proyecto nace de la aspiración de reducir las barreras comunicacionales de las personas señantes en interacciones cotidianas y de asistencia mediante herramientas informáticas. Resulta indispensable avanzar hacia esquemas de integración lingüística en los que la tecnología actúe como un puente transparente en ambos sentidos de la conversación: expresivo (señante $\rightarrow$ oyente) y receptivo (oyente $\rightarrow$ señante).

1.3. Problemática

La problemática central radica en la asimetría comunicacional presente en las plataformas de videoconferencia. Si bien las herramientas de transcripción automática de voz a texto han alcanzado un alto grado de adopción, resultan insuficientes como solución de accesibilidad integral: para diversas personas sordas, el texto escrito opera como una segunda lengua con reglas gramaticales ajenas, e incluso genera barreras de acceso para quienes no se encuentran alfabetizados en la lengua oral.

En el extremo expresivo, la comunicación mediante LSA carece de soporte nativo en el software de videollamadas comercial. La falta de módulos capaces de capturar e interpretar la LSA hacia voz o texto audible fuerza la dependencia de interpretación humana presencial o restringe la interacción al chat escrito.

A esta brecha se añade la especificidad lingüística local: los modelos de visión y lenguaje desarrollados internacionalmente se encuentran enfocados mayoritariamente en la Lengua de Señas Americana (ASL). Las técnicas y arquitecturas neuronales resultan reutilizables, pero los modelos preentrenados y conjuntos de datos de ASL no se extrapolan directamente a la LSA debido a diferencias léxicas, dactilológicas y sintácticas, por lo que es necesaria la construcción de un corpus propio y de modelos adaptados al contexto argentino.

1.4. Alcance y limitaciones

El sistema desarrollado comprende un prototipo funcional de traducción bidireccional en tiempo pseudo-real concebido como una extensión para navegadores basados en la tecnología Chromium (como Google Chrome, Microsoft Edge, Brave, Opera o Vivaldi), articulada con un motor de ejecución en segundo plano denominado ILSA (*Intérprete de LSA*).

Para abordar la interpretación de la LSA se introduce el concepto de **glosa**: representación escrita en letras mayúsculas asignada a una seña para identificarla, y analizar su gramática sin constituir una traducción literal término a término.

El sistema opera a través de dos canales comunicacionales integrados:

- **Canal directo (Señante → Oyente):** En el cliente, el sistema toma señas aisladas de la cámara, extrae coordenadas articulares y clasifica cada glosa. Una seña empieza cuando las manos entran en el cuadro y termina cuando salen. La frase se cierra tras 4 segundos sin manos en el cuadro. Una sola glosa léxica, un deletreo de letras o una seguidilla de dígitos se resuelven en ILSA y no consultan la nube. El resto viaja como texto de glosas al servicio semántico, que devuelve una oración en español rioplatense. Con la salida Subtítulo, la oración queda en el video. Con Audio o Ambos, la voz suena para quien seña y también para el resto de los participantes de la llamada. Si el servicio no responde, ILSA muestra las glosas aceptadas en texto, sin rearmar la oración.

- **Canal inverso (Oyente → Señante):** La voz del oyente se transcribe en su computadora, o el sistema toma el texto del chat. La oración va a la nube, que la reordena según el orden manual de este prototipo. El cliente dibuja el clip de pose de cada glosa sobre un esqueleto cuando el archivo existe. Si la glosa no tiene archivo, ILSA pide cada letra y la dibuja con el alfabeto manual. Ese deletreo no asegura que el interlocutor comprenda el significado de la palabra.

1.4.1. Delimitación del vocabulario y asesoramiento lingüístico

El alcance funcional del clasificador gestual está delimitado a un vocabulario objetivo de 100 señas de LSA (cuyo detalle y categorización se presentan en el Apéndice A). Para la selección y estructuración de este corpus se contó con la colaboración y asesoramiento técnico de Miriam Rolls, profesora e intérprete de LSA, quien participó en la delimitación del recorte léxico. El vocabulario se orientó hacia situaciones de interacción cotidiana y contextos de primera respuesta o emergencia (verbos de auxilio, identificación, parentesco

y datos temporales). Esas señas se distinguen sin la gesticulación facial. La validación garantizó la fidelidad lingüística de las señas seleccionadas para la construcción del corpus, sin que esto constituya una evaluación formal de usabilidad ni un ensayo clínico con usuarios finales en entornos reales de emergencia.

1.4.2. Límites operativos y comportamiento ante entradas no contempladas

El prototipo presenta las siguientes delimitaciones operativas:

- **Delimitación de señas y de frases:** Una seña empieza cuando las manos entran en el cuadro y termina cuando salen. La frase termina cuando pasan 4 segundos sin manos en el cuadro.
- **Mano hábil:** ILSA muestra dos botones, diestro y zurdo. Diestro aparece seleccionado. La persona confirma esa opción o pasa a zurdo: siempre hay una mano elegida. Con zurdo, el vector se espeja en horizontal y se cruzan las dos manos, porque el modelo se entrenó con mano derecha.
- **Rasgos no manuales y marcas manuales:** El clasificador excluye la malla facial y no trata la boca, las cejas ni la cabeza como rasgo gramatical. Una pregunta, una negación o un tiempo solo entran al enunciado si se producen con una seña o una marca manual. El prototipo no recupera un significado que exista únicamente en la información que descarta.
- **Predicción, glosa incorporada e indicador del HUD:** Una predicción es la clase que devuelve el clasificador, con su confianza. Una glosa incorporada es la que quedó en la frase. Ver el nombre en el HUD no significa que vaya a transmitirse. El color informa el resultado:
  - **Cian, «En el mensaje: HOLA»:** la confianza es de al menos 0,75 y la glosa quedó en la frase.
  - **Coral, «Poca confianza: HOLA»:** el clasificador reconoció la seña, pero la confianza quedó bajo 0,75 y no se agregó.
  - **Gris, «No se agregó: HOLA»:** la confianza alcanzó 0,75 y un filtro de repetición la dejó fuera. Ese filtro rechaza la repetición inmediata de una glosa léxica, admite la repetición en los dígitos y admite hasta dos letras iguales seguidas.

  Solo el cian entra al enunciado. Una seña ausente del vocabulario puede parecerse a una clase conocida y salir con confianza de al menos 0,75. En ese caso el HUD la muestra en cian y la glosa incorrecta entra a la frase.
- **Reproducción del avatar:** El dibujo está garantizado para el clip cuyo archivo existe, hasta 24 archivos por enunciado. Si una glosa no tiene archivo, se deletrea letra por letra, y ese deletreo no asegura la comprensión del significado. Lo que supera los 24 archivos queda en texto y no se anima. Si el servidor de movimientos no responde, las glosas quedan en texto y el clip queda vacío. El avatar mantiene, además, una cola de hasta 20 animaciones, una por oración. Una oración nueva espera su turno y no interrumpe la animación en curso.

- **Resiliencia ante fallas de red:** Si el traductor no responde, ILSA conserva las glosas ya aceptadas y las muestra en texto, sin rearmar la oración.
- **Arranque:** Encender, Reiniciar o Apagar vuelven a la pantalla de carga. Ahí ILSA exige que respondan el traductor semántico y el servidor de poses. Si alguno falla, queda el botón Reintentar y el panel principal no se abre.

1.4.3. Infraestructura de despliegue

En la red, el alcance incluye el diseño de una arquitectura elástica, con contenedores y balanceo de carga, para varias videollamadas a la vez. Las pruebas no despliegan ese esquema. La evaluación usa una instancia de Cloud Run por servicio, en CPU.

Capítulo 2

Objetivos

2.1. Objetivo general

Desarrollar un prototipo de sistema de traducción bidireccional entre la LSA y el español rioplatense con procesamiento en tiempo pseudo-real, diseñado para operar sobre plataformas de videoconferencia mediante una arquitectura híbrida cliente-nube, con el fin de mitigar la asimetría en la comunicación expresiva y receptiva, lo que ayudaría a fomentar la autonomía de las personas sordas en la interacción con interlocutores oyentes no señantes.

2.2. Objetivos específicos

- Implementar un módulo local de visión computacional y clasificación gestual basado en la extracción de puntos de referencia corporales para el reconocimiento de un vocabulario de 100 señas de LSA.
- Diseñar e integrar el servicio semántico del canal directo (LSA → Español) para transformar secuencias de glosas aisladas en oraciones fluidas y gramaticalmente correctas en español rioplatense.
- Diseñar e integrar el servicio semántico del canal inverso (Español → LSA) para reestructurar oraciones del español en la secuencia manual de glosas que fija este prototipo, dentro del orden de la Sección 3.1.2.
- Implementar un módulo de visualización gestual inversa mediante un avatar capaz de animar el clip de cada glosa con archivo y de deletrear los términos ausentes, con el límite de garantía de la Sección 1.4.2.
- Diseñar una arquitectura escalable en la nube para el aprovisionamiento concurrente de los servicios, lo que desacopla el procesamiento del hardware cliente.
- Resguardar la privacidad de los datos biométricos de los usuarios al ejecutar la extracción de características y la clasificación de forma local en el dispositivo cliente, y, por consiguiente, limitar la transmisión hacia la nube a cargas puramente textuales.
- Desarrollar la arquitectura de software de una extensión para navegadores basados en la tecnología Chromium, articulada con un motor de ejecución en segundo plano (ILSA).

- Evaluar el clasificador con un script local, seña por seña, según la Sección 7.2.1. Evaluar la fidelidad semántica de los dos canales según la Sección 3.3, aparte de la comprensión del avatar. Medir la latencia por etapas, según la Sección 3.1.3, y el consumo de hardware.

Capítulo 3

Marco teórico, estado del arte y metodología

3.1. Marco teórico

3.1.1. Componentes articulatorios y estructura fonológica de la LSA

La articulación de la LSA se sostiene sobre la interacción simultánea de dos grandes conjuntos de parámetros lingüísticos:

- **Rasgos manuales:** Comprenden la configuración o forma de la mano, la ubicación o punto de articulación en el espacio corporal, la orientación de la palma y la trayectoria cinemática [2].
- **Rasgos no manuales:** Incluyen la gesticulación facial (movimiento de cejas, boca y mirada), la inclinación del torso y la postura del cuello [2]. Estos elementos cumplen un rol determinante para matizar la intensidad de una acción, marcar interrogaciones o delimitar la polaridad en enunciados complejos.

Desde la perspectiva de la fonología gestual, la ejecución temporal de toda seña o glosa se estructura en tres fases secuenciales delimitadas en la literatura lingüística [3]:

- **Preparación (*Movement*):** Transición cinemática desde la postura de reposo o neutra hasta el punto de articulación inicial.
- **Núcleo (*Hold/Stroke*):** Fase central en la cual la configuración manual, la localización espacial y el movimiento otorgan el valor fonológico distintivo a la glosa.
- **Retracción (*Retraction*):** Retorno cinemático desde el término de la seña hacia la postura neutra de reposo.

3.1.2. Estructura lingüística de la LSA y orden manual del prototipo

La LSA y el español rioplatense no comparten la misma gramática. Las afirmaciones generales sobre la lengua y la regla de trabajo de este prototipo van separadas.

Sobre la LSA, la bibliografía de referencia sostiene tres puntos. La organización *Topic-Comment* presenta un tema y después lo que se dice de ese tema [2]. Esa organización no equivale, por sí sola, a un orden fijo de todas las palabras. La lengua no usa los artículos, las preposiciones gramaticales abstractas ni los verbos copulativos del español [2, 4].

Este prototipo no implementa esa gramática completa. El clasificador no toma la boca, las cejas ni la cabeza como rasgos. Una pregunta, una negación o un tiempo solo se recuperan si la persona los produce con una seña manual. La ecuación siguiente es la regla simplificada con la que el prototipo ordena las glosas, no una descripción de toda la LSA:

$$\begin{aligned}\text{Glosas del prototipo} &\rightarrow \text{Tiempo} \rightarrow \text{Lugar} \rightarrow \text{Sujeto} \\ &\rightarrow \text{Objeto} \rightarrow \text{Verbo} \rightarrow \text{Negación o pregunta}\end{aligned} \tag{3.1}$$

Debido a estas divergencias morfosintácticas, la equivalencia entre ambas lenguas no admite un mapeo léxico directo o traducción término a término. Quedan fuera la pregunta o la negación hechas solo con la cara, y cualquier significado que no tenga una seña manual en este recorte.

3.1.3. Sistemas de tiempo real y tiempo pseudo-real

El tiempo pseudo-real, en este prototipo, es una suma de esperas distintas. Cada una se cuenta desde un evento propio, y el servidor no las mezcla.

- **Pausa.** Empieza cuando las manos salen del cuadro y termina a los 4 segundos si no vuelven. Cierra la frase. No es tiempo del modelo.
- **Cola.** Empieza cuando la solicitud llega al servidor y termina cuando el modelo la toma. El servidor informa ese tiempo aparte de la inferencia.
- **Inferencia.** Empieza cuando el modelo toma la solicitud y termina cuando devuelve el texto. Es el único tramo al que el requerimiento RNF-01 exige un máximo de 2 segundos en banda ancha.
- **Entrega.** Empieza cuando el texto vuelve al cliente. En el canal directo incluye el subtítulo y la voz. En el canal inverso incluye la descarga de los movimientos, el dibujo y la salida del video ya compuesto hacia la llamada.

El ciclo que espera el interlocutor suma las cuatro. La meta de 2 segundos no cubre ese ciclo: la pausa, sola, ya dura 4 segundos. El procedimiento que mide cada tramo está en la Sección 7.4.2.

3.1.4. Visión computacional y reconocimiento basado en esqueletos

El reconocimiento de señas a partir de flujo de video crudo opera, en cada fotograma, sobre millones de valores cromáticos. Un cuadro de 1920×1080 píxeles en espacio RGB contiene:

$$1920 \times 1080 \times 3 = 6\,220\,800 \tag{3.2}$$

escalares. Esta representación resulta altamente sensible a las variaciones de iluminación, al fondo y a la vestimenta del usuario. El reconocimiento basado en esqueletos reemplaza

esta representación densa mediante la clasificación de un conjunto reducido de coordenadas articulares, denominadas *landmarks*.

Para la estimación postural se utiliza *MediaPipe Holistic*, una arquitectura de percepción distribuida para dispositivos de borde [5, 6]. Este marco no constituye un detector único, sino una serie de modelos preentrenados encadenados, que se ejecuta de forma local en el cliente sin transmitir la imagen cruda. El primer modelo, *BlazePose*, localiza al sujeto y estima 33 puntos de pose corporal [7]. Dichos puntos abarcan la estructura facial gruesa, el torso y las extremidades. A partir de las coordenadas de las muñecas, el pipeline delimita una región de interés (*ROI*) para cada mano, donde el modelo específico de manos (*hand*) infiere 21 puntos tridimensionales por mano (la muñeca y cuatro articulaciones por dedo) [8].

Asimismo, los datos cinemáticos de referencia gestual se archivan en un formato binario especializado con extensión `.pose` [9]. La especificación del formato y su esquema de serialización provienen de la biblioteca *pose-format*, desarrollada para el almacenamiento y manipulación de datos posturales en lenguas de señas, la cual organiza los puntos en un encabezado de metadatos globales y un tensor multidimensional de coordenadas espacializadas.

3.1.5. Modelado de secuencias temporales y atención

La clasificación gestual requiere procesar secuencias temporales de coordenadas articulares para asignar una clase o glosa. La aplicación de modelos recurrentes convencionales, tales como las redes LSTM, limita la compresión de la información histórica en un estado oculto secuencial, lo cual puede diluir los instantes iniciales del gesto al alcanzar la pose final.

El mecanismo de atención elimina esta restricción al permitir que cualquier instante temporal consulte de manera directa la totalidad de la secuencia en una única operación matricial [10]. Para cada entrada, el modelo proyecta tres vectores: consulta (Q), clave (K) y valor (V). La respuesta atencional se calcula mediante la relación:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{3.3}$$

Los símbolos de esa ecuación se leen así:

- Q, K y V son las matrices de consulta, clave y valor. Cada fila es un instante de la secuencia.
- $K^\top$ es la transpuesta de K. El producto $QK^\top$ compara la consulta de cada instante con la clave de todos los instantes.
- d_k es la cantidad de componentes de cada vector de clave, igual a la de cada vector de consulta. No es el largo de la secuencia ni la cantidad de clases.
- $\sqrt{d_k}$ divide esos productos. Sin esa división, un d_k grande empuja al softmax hacia un solo instante.
- softmax convierte cada fila en pesos que suman 1.
- El producto por V suma los vectores de valor con esos pesos.

Este cálculo habilita al clasificador a ponderar la pose sostenida durante el núcleo por encima de las transiciones.

3.1.6. Modelos de lenguaje pequeños y adaptación *LoRA*

Para la traducción semántica bidireccional se emplean modelos de lenguaje pequeños (SLM). La adaptación de estos modelos se ejecuta mediante la técnica de Adaptación de Bajo Rango (*LoRA*) [11], la cual congela la matriz de pesos preentrenados $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ e introduce dos matrices entrenables de bajo rango $\mathbf{B} \in \mathbb{R}^{d\times r}$ y $\mathbf{A} \in \mathbb{R}^{r\times k}$, con rango $r \ll \min(d, k)$. La actualización lineal adopta la forma:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{BA}) \tag{3.4}$$

donde α actúa como un factor de escala constante. La figura dibuja el caso cuadrado $k = d$: $\mathbf{B}$ mide $d \times r$ y $\mathbf{A}$ mide $r \times d$. El producto BA recupera el tamaño de ΔW.

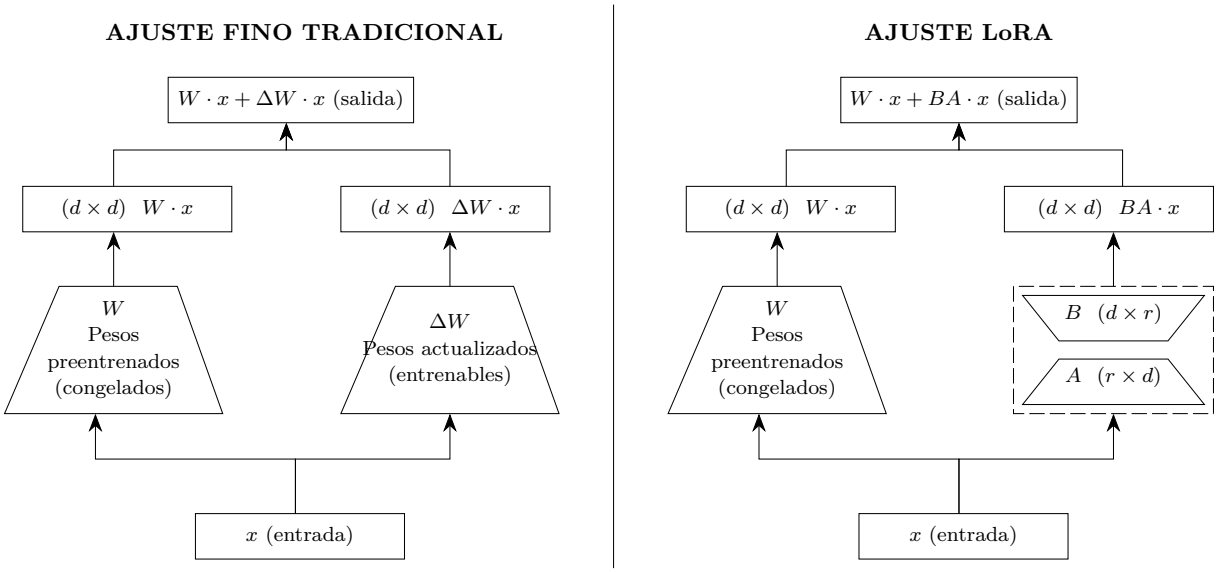


Figura 3.1: Figura del ajuste. A la izquierda, ΔW conserva el tamaño $d \times d$. A la derecha, el caso cuadrado $k = d$: B mide $d \times r$, A mide $r \times d$ y el producto BA vuelve al tamaño de ΔW.

Tras el ajuste, los adaptadores se fusionan con la base. Ese resultado sigue en 16 bits. Hay que distinguir tres cosas. El formato de archivo es GGUF. La configuración de cuantización es el preset `Q4_K_M` de `llama.cpp` [12]. La ecuación de abajo es una reconstrucción simplificada de un peso dentro de un bloque.

El conversor parte cada tensor en superbloques de 256 pesos y, dentro, en bloques más cortos. Cada bloque guarda enteros de 4 bits junto con una escala s y un mínimo m:

$$\hat{w} = m + s\,q \tag{3.5}$$

donde q es el entero de ese peso. En la tabla del preset, el costo medio queda cerca de 5 bits por parámetro, frente a 16. Esa cuenta corresponde a `Q4_K_M`, no a una medición de calidad sobre las oraciones de este trabajo. La mayor parte de los tensores queda en `Q4_K`. El preset reserva `Q6_K` para `v_proj` y para `down_proj`. Las normalizaciones permanecen en 32 bits. Con ese tamaño de archivo, el traductor de las pruebas corre en CPU.

3.1.7. Estándares de captura web y arquitecturas de extensión web

La captura de video en el navegador se apoya en dos especificaciones. *Media Capture and Streams* define `getUserMedia()` [13]. Esa llamada entrega pistas de un dispositivo, por ejemplo la cámara. La misma especificación permite varias pistas asociadas a una misma fuente.

En el entorno de ejecución objetivo (Windows, Chrome y Google Meet), el acceso directo a la cámara mediante getUserMedia() falla con la excepción NotReadableError si el dispositivo físico ya está reservado por la videoconferencia. Esto ocurre porque el cliente de captura opera en modo de control exclusivo. Por este motivo, la arquitectura de ILSA omite la apertura independiente de la cámara y reutiliza el objeto MediaStream ya existente en la reunión. Asimismo, al desactivar la extensión, el sistema conserva los flujos activos y no invoca la función stop() sobre sus pistas.

El lienzo arma la imagen que sale a la llamada. Los fotogramas se leen de ese mismo flujo y el dibujo agrega el subtítulo o el esqueleto. `canvas.captureStream()`, de *Media Capture from DOM Elements* [14], construye un `MediaStream` a partir del lienzo donde esos dibujos ya están.

En el ámbito de las extensiones web bajo el estándar Manifest V3 [15], la Política de Seguridad de Contenido (CSP) prohíbe la ejecución de código dinámico en scripts de fondo. Para ejecutar los clasificadores y estimadores de visión computacional se implementa una arquitectura desacoplada en dos entornos aislados:

- **Documento en segundo plano (*offscreen*):** Entorno habilitado por la API `chrome.offscreen` [16] que provee contexto de DOM completo para coordinar la mensajería y gestionar lienzos desacoplados.
- **Entorno aislado (*sandbox*):** Entorno habilitado por el manifiesto [17] con CSP relajada que autoriza la instanciación de trabajadores web (*Web Workers*) dedicados al cómputo intensivo de inferencia.

3.2. Estado del arte

La comparación usa cuatro ejes: lengua, tipo de entrada, sentido de la traducción e integración en una llamada. El prototipo reconoce señas aisladas de LSA, a partir de los puntos de una cámara web, en los dos sentidos, dentro de Google Meet. No reconoce un discurso continuo sin corte entre señas.

Cuadro 3.1: Sistemas de referencia según lengua, entrada, sentido e integración.

Sistema	**Lengua y entrada**	**Sentido**	**Integración**
Diccionario INJU [18]	LSA, consulta léxica	No traduce una frase en llamada.	Índice en línea.
Señario de la CAS [19]	LSA, material pedagógico	Consulta estática.	Fuera de la videollamada.
LSApp [20]	LSA, ítem de diccionario o deletreo	Hacia la animación, en un solo sentido.	Aplicación móvil, sin reconocimiento.
Blessit y Línea 144 [21, 22]	LSA, video con una persona	Interpretación humana.	Videollamada sujeta al turno del operador.
LSA64 y redes recurrentes posteriores [23, 24]	LSA, video con guantes de colores	Reconocimiento de señas aisladas.	Laboratorio. Los guantes impiden el paso directo a una cámara web.
SignAll [25]	ASL, visión	Hacia inglés escrito.	Sin devolución en señas.
WLASL [26]	ASL, video	Reconocimiento léxico.	Corpus. No traduce oraciones ni trabaja en LSA.
Hand Talk [27]	Libras y ASL, texto	Hacia un avatar 3D.	Sin reconocimiento y sin LSA.
HETAH [28]	Español, oración escrita	Hacia síntesis gestual.	Sin reconocimiento de LSA dentro de una llamada.

Cada límite de la tabla está en la fuente citada: el diccionario no reconoce, LSA64 filma con guantes, SignAll no devuelve señas, WLASL no sale de la ASL léxica, y Hand Talk y HETAH parten del texto. Ninguna de esas fuentes describe el recorte de este prototipo, que es el reconocimiento aislado dentro de Meet.

3.3. Metodología de desarrollo

El proyecto se estructuró bajo un modelo de ciclo de vida iterativo e incremental [29], compuesto por 6 fases principales:

1. **Curación de datos y formalización léxica:** Delimitación de 100 señas de LSA en colaboración con un informante calificado; diseño del entorno de grabación; captura de clips en resolución 1080p; y generación del dataset de coordenadas vectoriales normalizadas.

2. **Entrenamiento y diagnóstico del clasificador gestual:** Prueba de ventanas de 8, 12, 16 y 24 fotogramas; búsqueda de hiperparámetros con Optuna [30]; diagnóstico de confusiones con el AUC de Mann-Whitney.

3. **Desarrollo de los módulos semánticos bidireccionales y diseño de despliegue:** Diseño de prompts; ajuste con LoRA sobre Qwen2.5 [31], SmolLM2 [32] y

Llama 3.2 [33], en el orden manual de la Sección 3.1.2; cuantización GGUF [12]; y evaluación de fidelidad según la Sección 3.3.

4. **Integración y renderizado gestual:** Acoplamiento del pipeline en el motor local ILSA mediante patrones de diseño desacoplados; comunicación con el servidor semántico; e intercepción de video en Google Meet mediante Canvas sintético y extensión Manifest V3 [15].

5. **Avatar e integración visual inversa:** Desarrollo del controlador del avatar para la representación esquelética de las glosas del canal inverso; implementación del subsistema de contingencia para deletreo manual dactilológico; e integración en la extensión.

6. **Fase de pruebas:** Medición de las etapas de la Sección 3.1.3, con el procedimiento de la Sección 7.4.2.

Fidelidad semántica La fidelidad semántica, en este trabajo, es la conservación de la proposición dentro de los mensajes que el prototipo admite: quién interviene, de qué se habla, en qué tiempo, si hay negación y si hay pregunta. La evaluación automática compara la salida con una referencia. Se registran cinco métricas porque cada una mira un fallo distinto. BLEU y ROUGE-L castigan una paráfrasis correcta y pueden premiar un texto que repite palabras pero cambia la persona, el tiempo, la negación o la pregunta. METEOR afloja esa exigencia cuando coincide la raíz. `bertscore_f1` acepta palabras distintas si sus vectores quedan cerca. `comet_score` mira también la entrada, no solo la referencia.

- **BLEU** [34]. Precisión de n-gramas: qué porción de las secuencias de la salida ya está en la referencia. Una salida corta recibe una penalización. No premia, con el mismo peso, las palabras de la referencia que la salida omitió.

- **ROUGE-L** [35]. La subsecuencia común más larga, respetando el orden. Dos textos pueden compartir palabras y aun así bajar si esas palabras no aparecen en el mismo orden.

- **METEOR** [36]. Alinea palabras sueltas. Cuenta la coincidencia exacta, la raíz y, si hay un léxico de sinónimos, el sinónimo. Junta precisión y recall, y baja si el texto queda partido en muchos fragmentos.

- `BERT Score` [37]. Compara el vector de cada token de la salida con los de la referencia, mediante un modelo de lenguaje preentrenado. El F1 de esa alineación sube cuando el sentido está cerca aunque las palabras no coincidan.

- `Comet Score` [38]. Un modelo entrenado para aproximar juicios humanos de traducción. Usa la entrada, la salida y la referencia. Estima si el sentido de la entrada llegó a la salida.

Un puntaje alto de solapamiento puede convivir con un cambio de significado, y una paráfrasis correcta puede puntuar bajo en BLEU o en ROUGE-L. `BERT Score` y `Comet Score` achican ese hueco. No lo cierran. El error de significado se revisa aparte, sobre esos mismos pares, y se informa en la Sección 7.3.3.

3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial

Herramientas empleadas. Durante las fases de desarrollo y redacción se utilizaron dos herramientas auxiliares:

- **Cursor:** Entorno de desarrollo integrado asistido por inteligencia artificial, utilizado para la navegación del repositorio, autocompletado sintáctico y asistencia en la edición de código en Python y JavaScript.
- **Gemini (Google):** Modelo de lenguaje utilizado como soporte de consulta técnica, facilitador en la asimilación de conceptos teóricos y asistencia en la revisión del documento. También redactó borradores de pares español–glosas para el corpus semántico, a partir de un prompt que fijaba el orden manual, el vocabulario y los fenómenos a cubrir. Cada par se leyó y se corrigió antes de entrar al archivo de entrenamiento.

Alcance y propósito de su utilización. La intervención de dichas herramientas fue con el propósito de:

- **Consultas técnicas y conceptuales:** Asistencia en la comprensión de aspectos específicos relativos a la cuantización GGUF Q4_K_M con `llama.cpp`, fundamentos de LoRA y esquemas de comunicación asíncrona en extensiones Manifest V3.
- **Asistencia en el desarrollo de software:** Sugerencias de refactorización de código y resolución de incompatibilidades sintácticas en el entorno Chrome Extensions.
- **Revisión de formato:** Corrección gramatical y adecuación de estilo académico del informe.

Límites y responsabilidad de autoría. Ninguna herramienta generativa reemplazó la labor intelectual, analítica ni experimental del proyecto. La concepción de la arquitectura híbrida, la recolección y curación del corpus de 100 señas, el diseño e iteración de las redes neuronales, el ajuste de los modelos de lenguaje y el análisis de resultados constituyen desarrollo de los autores, quienes asumen la responsabilidad técnica y académica por la totalidad del contenido. Los pares del corpus semántico se redactaron con asistencia de Gemini bajo un prompt definido por los autores. Ellos revisaron y aceptaron cada ejemplo antes de entrenar.

3.4. Herramientas y tecnologías empleadas

Cuadro 3.2: Conjunto de tecnologías del sistema integrado

Categoría	**Herramienta**	**Rol en el sistema**
Entorno y Lenguajes	Python 3.11, JavaScript	Lógica de backend, motor local, servidor semántico y extensión web.
Visión Computacional	MediaPipe Holistic, OpenCV	Extracción de coordenadas articulares corporales y manipulación de video.
Aprendizaje Profundo	PyTorch 2.6	Inferencia local del clasificador gestual.
Modelos de Lenguaje	Transformers, PEFT, Unsloth	Fine-tuning eficiente con adaptadores LoRA para traducción directa e inversa.
Inferencia de Lenguaje	`llama.cpp`, GGUF	Inferencia del traductor en CPU.
Comunicación y Backend	FastAPI, Uvicorn	Servidor de interfaz local (ILSA) y microservicio semántico desacoplado.
Servicio en la nube	Google Cloud Run	Instancia del servicio semántico empleada en las pruebas experimentales.
Integración Web	Chrome Extensions (MV3)	Intercepción de media mediante Canvas `captureStream()` en Google Meet.
Transcripción	Vosk	Transcripción local de audio a texto en el cliente.
Síntesis de Salida	Síntesis del navegador y avatar	Generación de voz sintética y dibujo esquelético del avatar gestual.
Avatar	PoseFormat	Conversión de metadatos y almacenamiento en formato binario `.pose`.

Capítulo 4

Análisis de requerimientos

4.1. Requerimientos funcionales

- **RF-01 — Captura y extracción local de landmarks:** Capturar el video del usuario señante en el cliente y extraer los 33 puntos de pose y los 21 de cada mano, en tres coordenadas. Cada punto resta el centro de los hombros y queda dividido por la distancia entre hombros.
- **RF-02 — Clasificación gestual temporal local:** Clasificar localmente secuencias fijas de coordenadas en una de las 100 clases de LSA soportadas y utilizar arquitecturas de aprendizaje profundo para reportar la glosa asignada y su confianza.
- **RF-03 — Traducción semántica directa remota (LSA → Español):** Enviar a la nube el texto de glosas de un enunciado y recibir una oración en español rioplatense. Una glosa léxica sola, un deletreo de letras o una seguidilla de dígitos se resuelven en ILSA y no salen a la nube.
- **RF-04 — Emisión hacia el oyente:** Con Subtítulo, mostrar la oración en el video. Con Audio o Ambos, reproducir la voz para quien señа y para el resto de los participantes de la llamada. En Ambos, el subtítulo y la voz salen juntos.
- **RF-05 — Captura del turno del interlocutor oyente:** Capturar la voz o texto del oyente en la videollamada durante su turno de intervención con el fin de ser procesada para su uso en el canal de retorno bidireccional.
- **RF-06 — Traducción semántica inversa remota (Español → LSA):** Enviar el texto en español al servicio semántico y recibir glosas en el orden manual de la Sección 3.1.2.
- **RF-07 — Representación visual mediante un avatar:** Dibujar el clip de cada glosa que tenga archivo, hasta 24 archivos por oración. Si falta el archivo, deletrear letra por letra. El deletreo no asegura la comprensión del significado. Lo que supere el tope queda fuera del dibujo y se conserva en texto. Si el servidor de movimientos no responde, el clip queda vacío y las glosas siguen en texto. Encolar cada oración: la cola admite hasta 20 animaciones, y una oración nueva espera su turno sin interrumpir la animación en curso.
- **RF-08 — Integración con Google Meet:** Inyectar el video procesado y los canales de comunicación dentro de la pestaña de videollamada.

- **RF-09 — Arranque del panel:** Al Encender, Reiniciar o Apagar, volver a la pantalla de carga. Abrir el panel principal solo si responden el traductor semántico y el servidor de poses. Si alguno falla, dejar el botón Reintentar y no abrir el panel.

4.2. Requerimientos no funcionales

- **RNF-01 — Latencia y tiempo de respuesta:** La pausa, la cola, la inferencia y la entrega se cuentan por separado, como fija la Sección 3.1.3. La pausa de cierre es de 4 segundos sin manos y no entra en la meta del modelo. La cola y la inferencia salen en campos distintos de la respuesta del servidor. La inferencia no debe superar los 2 segundos en banda ancha. El ciclo completo no tiene ese tope. Su medición está en la Sección 7.4.2.

- **RNF-02 — Datos que recibe cada componente:** Meet recibe el video ya compuesto (cámara, subtítulo y, en modo oyente, el esqueleto) y el audio de la llamada, incluida la voz sintética cuando la salida es Audio o Ambos. ILSA, en la misma computadora, recibe los puntos articulares y el audio local para transcribir. El traductor en la nube recibe solo texto: glosas u oración. El servidor de movimientos recibe el nombre de una glosa o las letras de un deletreo, y devuelve archivos. La prohibición de enviar imagen o video crudo rige para el traductor y para el servidor de movimientos. El tratamiento dentro de este alcance es ese reparto: la imagen de la cámara sale hacia Meet porque es la llamada, y no sale hacia los dos servicios de la nube.

- **RNF-03 — Robustez en reconocimiento en vivo:** El clasificador debe superar el 75 % de acierto en luz doméstica. Ese valor es una meta de reconocimiento.

- **RNF-04 — Desacoplamiento y escalabilidad de los servicios en la nube:** La arquitectura del backend semántico y del servidor de archivos .pose deben estar concebidas para operar de forma desacoplada y elástica, lo que permita sus despliegues en infraestructuras con auto-escalado horizontal a través de interfaces REST tipadas.

- **RNF-05 — Usabilidad y no invasividad:** La interfaz sobre la videollamada debe ofrecer indicadores visuales de estado que no ocluyan la visibilidad del interlocutor para mitigar la brecha comunicacional receptiva y expresiva.

4.3. Restricciones

- **RC-01 — Alcance léxico:** El vocabulario reconocido por el clasificador está acotado a 100 señas validadas por profesionales de LSA, enmarcadas en las pautas pedagógicas y lingüísticas vigentes.

- **RC-02 — Despliegue en la nube del prototipo:** Las pruebas no despliegan la arquitectura elástica del servicio semántico o del servicio de archivos .pose. La validación usa Cloud Run, de Google Cloud.

- **RC-03 — Compatibilidad de plataforma cliente:** El empaquetado del motor local (ILSA) y la extension estan diseñados para el sistema operativo Windows y navegadores basados en Chromium bajo las especificaciones de seguridad vigentes.

- **RC-04 — Dependencia de visión:** El clasificador depende del correcto funcionamiento de MediaPipe Holistic para la extraccion de landmarks.
- **RC-05 — Recursos del cliente objetivo:** El componente local del prototipo debe ser capaz de operar en laptops estándar (procesadores de 4 núcleos y 8 GB de memoria RAM, sin placa gráfica dedicada).

4.3.1. Especificación del entorno de ejecución cliente

El cuadro siguiente propone un equipo mínimo y uno con más margen. No informa una prueba sobre cada modelo de procesador ni sobre cada navegador de la lista. El nombre comercial de una familia cubre piezas distintas: el criterio operativo es el número de núcleos, el soporte AVX2 y el piso de reloj, no la etiqueta Core i3 o Ryzen 3.

El mínimo se elige para que convivan tres cargas: la videollamada, la extracción de puntos en el navegador y la clasificación de ILSA en CPU [15]. De ahí salen cuatro núcleos, 8 GB de RAM, gráficos con WebGL para el lienzo, disco de estado sólido, cámara de 720p y banda ancha. Chrome 115 o posterior es la versión en la que se comprobó Manifest V3. Windows 10/11 de 64 bits es el sistema del empaquetado de ILSA.

La fila recomendada es una propuesta de holgura: más núcleos, más memoria y cámara de 1080p. No es una configuración medida. Edge, Brave, Opera y Vivaldi comparten motor Chromium, y por eso se proponen. Este trabajo no informa una pasada en cada uno: las políticas y las APIs pueden diferir. La comparación entre la máquina de desarrollo y la máquina objetivo queda en la Sección 7.4.3.

Cuadro 4.1: Requisitos de hardware y software del dispositivo cliente

Componente	**Requisitos mínimos (Dispositivo base)**	**Requisitos recomendados**
Procesador (CPU)	Intel Core i3 (10ª gen.) / AMD Ryzen 3 (serie 3000) (4 núcleos / 4 hilos, base $\geq$ 2,0 GHz, soporte AVX2).	Intel Core i5 (11ª gen.+) / AMD Ryzen 5 (serie 4000+) (6 núcleos / 12 hilos).
Memoria RAM	8 GB DDR4 ($\geq$ 3,0 GB libres para el entorno de ejecución).	16 GB DDR4 / DDR5.
Gráficos (GPU)	Gráficos integrados (Intel UHD Graphics 620 / AMD Radeon Vega 3) con soporte WebGL.	Intel Iris Xe / AMD Radeon 600M+ o GPU dedicada básica.
Almacenamiento	Disco de estado sólido (SSD) con 2 GB de espacio libre.	SSD NVMe con $\geq$ 5 GB libres.
Cámara web	Resolución 720p (1280×720) a 24/30 FPS.	Resolución 1080p a 30 FPS.
Conectividad	Conexión de banda ancha ($\geq$ 10 Mbps bajada / $\geq$ 3 Mbps subida, latencia $<$ 80 ms).	Conexión por fibra óptica ($\geq$ 50 Mbps).
Entorno de software	Windows 10/11 (64 bits), Google Chrome 115+ (soporte Manifest V3).	Windows 11 (64 bits), Google Chrome última versión estable.

Condiciones de funcionamiento La CPU debe sostener a la vez la videollamada, MediaPipe en WebAssembly y el clasificador en CPU. El informe no asigna núcleos por carga ni un tiempo de clasificación en milisegundos. Las condiciones que importan, en el mínimo propuesto, son: 4 núcleos con AVX2, 8 GB de RAM con memoria libre para el proceso, WebGL, Windows 10/11 de 64 bits y Chrome 115 o posterior.

4.4. Casos de uso

4.4.1. Casos de uso esenciales en formato breve

- **CU-01 — Conversar con otra persona (Modo Sordo):** El usuario sordo seña frente a la cámara durante la videollamada. El sistema clasifica las señas en el equipo. Según el enunciado, arma el texto en ILSA o consulta el servicio semántico. Con Subtítulo, la oración queda en el video. Con Audio o Ambos, la voz suena para todos los participantes de la llamada.
- **CU-02 — Mano hábil:** ILSA muestra diestro y zurdo, con diestro ya seleccionado. La persona confirma diestro o pasa a zurdo. Con zurdo, el vector se espeja en horizontal y se cruzan las dos manos durante esa sesión.
- **CU-03 — Configurar canal de salida:** En el panel de la extensión, el usuario señante elige la salida en Meet: Subtítulo, Audio o Ambos. Si no toca el control, queda Subtítulo.
- **CU-04 — Traducir español a LSA mediante Avatar (Modo Oyente):** En el flujo básico, el usuario oyente habla e ILSA transcribe esa voz. El texto del chat es un flujo alternativo del mismo caso: ILSA toma ese texto y no lo transcribe. El sistema pide las glosas al servicio semántico. El avatar encola la animación, hasta 20 oraciones en espera, dibuja cada clip que existe, hasta 24 archivos, y deletrea lo que no tiene archivo. El deletreo no asegura la comprensión del significado.
- **CU-05 — Entrar al panel:** Encender, Reiniciar o Apagar vuelven a la pantalla de carga. El panel se abre si responden el traductor semántico y el servidor de poses. Si alguno falla, queda Reintentar.

4.4.2. Casos de uso expandidos

Identificador	**CU-01: Conversar con otra persona (Modo Sordo)**		
Actor principal	Usuario señante		
Precondiciones	El usuario debe estar unido a una llamada.		
Postcondiciones	El interlocutor lee el subtítulo en el video. Con Audio o Ambos, la voz suena para quien señа y para el resto de los participantes.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Realiza las señas correspondientes a lo que quiere comunicar	
	2		El sistema clasifica la seña. Si la confianza es de al menos 0,75 y el filtro de repetición la admite, el HUD la muestra en cian («En el mensaje: HOLA») y esa glosa queda en la frase.
	3	Si necesita otra seña, vuelve al paso 1. Si no, espera 4 segundos sin manos en el cuadro.	
	4		Si el enunciado es una glosa léxica sola, un deletreo o dígitos, ILSA arma el texto en el equipo. En los demás casos envía las glosas al servicio semántico. Con Subtítulo, deja la oración en el video. Con Audio o Ambos, la voz suena para quien señа y para el resto de los participantes.
Extensiones (flujos alternativos)	**1a.** Las manos no entran al cuadro: no empieza ninguna seña y no hay plazo que contar. **2a.** Confianza bajo 0,75: el HUD muestra la predicción en coral, «Poca confianza: HOLA». Esa predicción no entra en la frase y no se envía. Quien señа puede repetirla antes de los 4 segundos sin manos. **2b.** Confianza de al menos 0,75 y repetición no admitida: el HUD muestra gris, «No se agregó: HOLA». La predicción no entra en la frase. **2c.** Seña ajena al vocabulario con confianza de al menos 0,75: el HUD la muestra en cian y la glosa incorrecta entra. No hay rechazo automático. **4a.** El traductor no responde: se conservan las glosas aceptadas, en texto, y no se rearma la oración. La recuperación es volver a señar cuando el servicio responde.		

Identificador	**CU-02: Mano hábil**		
Actor principal	Usuario señante		
Precondiciones	ILSA está en el panel principal. El control de mano muestra diestro seleccionado.		
Postcondiciones	La sesión queda con la mano elegida. Con zurdo, el vector se espeja y se cruzan las manos.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Ve diestro ya seleccionado y lo confirma.	
	2		Conserva la mano derecha. El vector no se espeja.
Extensiones (flujos alternativos)	**1a.** Selecciona zurdo: ILSA espeja el eje horizontal y cruza las dos manos, porque el modelo se entrenó con mano derecha. Conserva esa mano durante la sesión.		
Requisitos especiales	-		

Identificador	**CU-03: Configurar canal de salida del oyente**		
Actor principal	Usuario señante		
Precondiciones	El usuario debe estar unido a una llamada.		
Postcondiciones	La salida queda en subtítulo, en voz para todos los participantes, o en ambas. Si no hay elección, queda Subtítulo.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Abre el panel de la extensión.	
	2		Muestra tres opciones bajo «Salida en Meet»: Subtítulo, Audio y Ambos.
	3	Elige una opción.	
	4		Aplica esa salida. Subtítulo deja el texto en el video y no emite voz. Audio o Ambos hacen sonar la voz para quien señа y para el resto de los participantes.
Extensiones (flujos alternativos)	**3a.** No hay elección: se conserva Subtítulo. No hay voz. La recuperación es abrir el panel y elegir Audio o Ambos.		
Requisitos especiales	-		

<table>
<tr><td>Identificador</td><td colspan="3">CU-04: Traducir español a LSA mediante Avatar (Modo Oyente)</td></tr>
<tr><td>Actor principal</td><td colspan="3">Usuario oyente</td></tr>
<tr><td>Precondiciones</td><td colspan="3">El usuario está unido a una llamada y la extensión está en modo oyente.</td></tr>
<tr><td>Postcondiciones</td><td colspan="3">El video muestra el esqueleto de los clips que existen, hasta 24 archivos por oración. La animación entra en una cola de hasta 20 oraciones y no corta la que está en curso. El deletreo no asegura la comprensión del significado. El resto queda en texto.</td></tr>
<tr><td rowspan="5">Escenario Principal de éxito (flujo básico)</td><td></td><td>Usuario oyente</td><td>Sistema</td></tr>
<tr><td>1</td><td>Habla durante la llamada.</td><td></td></tr>
<tr><td>2</td><td></td><td>Transcribe esa voz en la misma computadora.</td></tr>
<tr><td>3</td><td></td><td>Envía la oración al servicio semántico y recibe las glosas.</td></tr>
<tr><td>4</td><td></td><td>Encola la animación. Si hay otra en curso, la nueva espera: la cola admite hasta 20. Dibuja el clip de cada glosa que tiene archivo, hasta 24. Si una glosa no tiene archivo, dibuja sus letras una por una. Ese deletreo no asegura la comprensión del significado.</td></tr>
<tr><td>Extensiones (flujos alternativos)</td><td colspan="3">1a. Escribe en el chat: ILSA toma ese texto, sin transcribirlo, y el flujo sigue en el paso 3.
3a. Si el servicio semántico no responde: no hay glosas nuevas, la cámara sigue y el indicador informa que la secuencia no se obtuvo. Se conserva el video anterior. La recuperación es repetir el turno cuando el servicio responde. No hay un plazo distinto del de la llamada.
4a. Falta el archivo de una glosa: se deletrea y el significado no queda garantizado.
4b. El enunciado supera 24 archivos: se dibujan los primeros 24 y el resto queda en texto.
4c. El servidor de movimientos no responde: las glosas quedan en texto, el clip queda vacío y la recuperación es repetir el turno.</td></tr>
<tr><td>Requisitos especiales</td><td colspan="3">-</td></tr>
</table>

Identificador	**CU-05: Entrar al panel**		
Actor principal	Usuario		
Precondiciones	ILSA está instalado en la computadora.		
Postcondiciones	Si responden el traductor semántico y el servidor de poses, se abre el panel principal.		
Escenario Principal de éxito (flujo básico)		**Usuario**	**Sistema**
	1	Pulsa Encender, Reiniciar o Apagar.	
	2		Vuelve a la pantalla de carga y consulta el traductor semántico y el servidor de poses.
	3		Ambos responden. Abre el panel principal.
Extensiones (flujos alternativos)	**2a.** El traductor o el servidor de poses no responde: queda el botón Reintentar y el panel principal no se abre. La recuperación es pulsar Reintentar, que repite la consulta de los dos servicios.		
Requisitos especiales	-		

Capítulo 5

Diseño del Sistema

5.1. Arquitectura del sistema

El diseño adopta una arquitectura híbrida desacoplada. La extracción de puntos articulares y la clasificación gestual se ejecutan en el equipo del usuario, mientras que el módulo de traducción semántica y servidor de poses operan como un servicio remoto distribuido.

5.1.1. Justificación de la partición y modelo de privacidad

La ejecución local de la extracción y clasificación gestual garantiza que la señal de video cruda y las coordenadas articulares no abandonen el equipo del usuario. La demanda de memoria RAM y capacidad de cómputo de los modelos de lenguaje exige situar el componente de traducción semántica en un servicio remoto en la nube. Hacia dicho servicio remoto se transmite únicamente la secuencia textual de glosas (en el canal directo) o la oración en español (en el canal inverso), sin transferir información biométrica o de video.

5.1.2. Flujo integral de datos entre componentes

La interacción entre el navegador, el motor local y los servicios remotos se organiza según la estructura representada en la Figura 5.1. El procesamiento local gestiona la captura y clasificación, mientras que la red transmite exclusivamente mensajes en formato JSON.

Para detallar la secuencia de intercambio de mensajes, las Figuras 5.2 y 5.3 ilustran el comportamiento de los canales directo e inverso respectivamente.

5.2. Diseño de contratos de API REST

La comunicación entre el cliente local y los microservicios se rige por contratos JSON estructurados.

5.2.1. API local del motor ILSA

El motor ILSA expone una interfaz HTTP en el bucle local. La extensión web actúa como cliente y el proceso local como servidor. El esquema general de componentes internos se representa en la Figura 5.4.

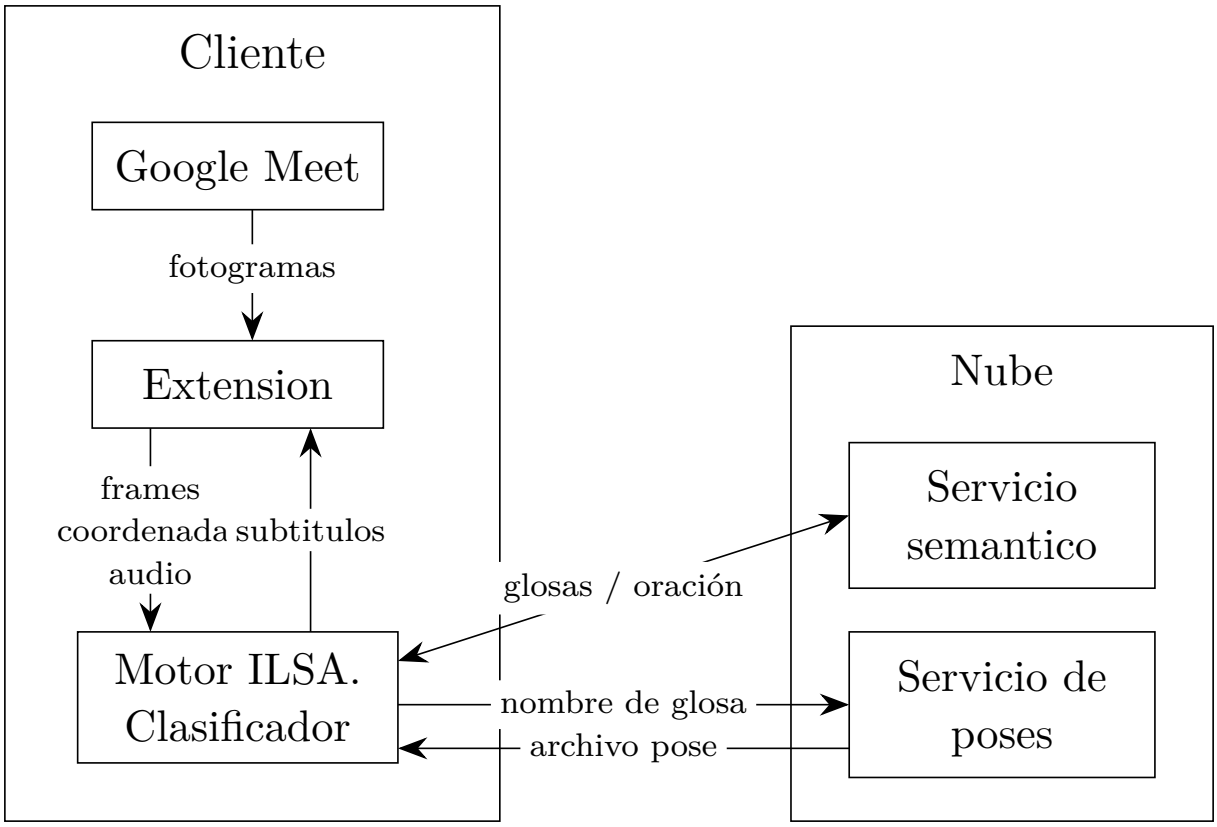


Figura 5.1: Diagrama de la arquitectura general del sistema.

El Cuadro 5.1 resume las rutas expuestas por el motor local.

5.2.2. API del servicio semántico remoto

El microservicio remoto en la nube expone contratos REST sobre HTTPS para resolver la traducción semántica desacoplada. El Cuadro 5.2 resume los endpoints principales.

5.2.3. API del servicio de archivos del avatar

El subsistema de representación gestual de salida consulta un repositorio de archivos cinemáticos binarios. El Cuadro 5.3 detalla las rutas disponibles.

5.3. Diseño de la representación gestual inversa

El canal inverso procesa la lista de glosas devuelta por el traductor e infiere la secuencia cinemática correspondiente. Cada glosa se asocia a un clip de referencias cinemáticas previamente parametrizado.

Cuadro 5.1: Endpoints de la API local del motor ILSA (`http://127.0.0.1:8765`).

Método	**Ruta**	**Contrato**	**Códigos**
GET	`/health`	Disponibilidad del proceso, del modo, del clasificador y del traductor remoto.	200
GET	`/config`	Umbrales de captura del modo en curso.	200, 503
GET	`/state`	Búfer de glosas, último texto y estado del traductor.	200, 503
GET	`/metrics`	Conteos y tiempos de la sesión en curso.	200
POST	`/session`	Apertura de sesión. El cuerpo fija la lateralidad mediante `left_handed`.	200, 503
POST	`/sign`	Ingesta de hasta 60 fotogramas para clasificación en modo señante.	200, 400, 503
POST	`/stt`	Transcripción de audio WAV en Base64 a texto en modo oyente.	200, 400, 502
POST	`/hearing`	Envío de oración y recepción de animación en modo oyente.	200, 400, 503
POST	`/activity`	Reinicio del temporizador de pausa de inactividad ($\sim 4\,\mathrm{s}$).	200, 503
POST	`/utterance/end`	Cierre explícito del enunciado y disparo de traducción.	200, 503
GET	`/mediapipe/{name}`	Despacho de recursos estáticos del estimador visual.	200, 400, 404
GET	`/download/exe`	Entrega del ejecutable del motor local.	200, 404

Cuadro 5.2: Endpoints del servicio semántico remoto.

Método	Ruta	Función
GET	`/health`	Verificación de disponibilidad de los modelos de lenguaje en memoria.
POST	`/sign`	Canal directo: traducción de glosas LSA a oración en español rioplatense.
POST	`/hearing`	Canal inverso: traducción de oración en español a glosas en estructura *Topic-Comment*.

Cuadro 5.3: Endpoints del servicio de archivos del avatar.

Método	Ruta	Función
GET	`/health`	Disponibilidad del proceso de archivos.
GET	`/tposes/{glosa}.pose`	Entrega del flujo binario cinemático correspondiente a la glosa solicitada.
GET	`/fingerspell`	Solicitud de clips del alfabeto manual para deletreo dactilológico.

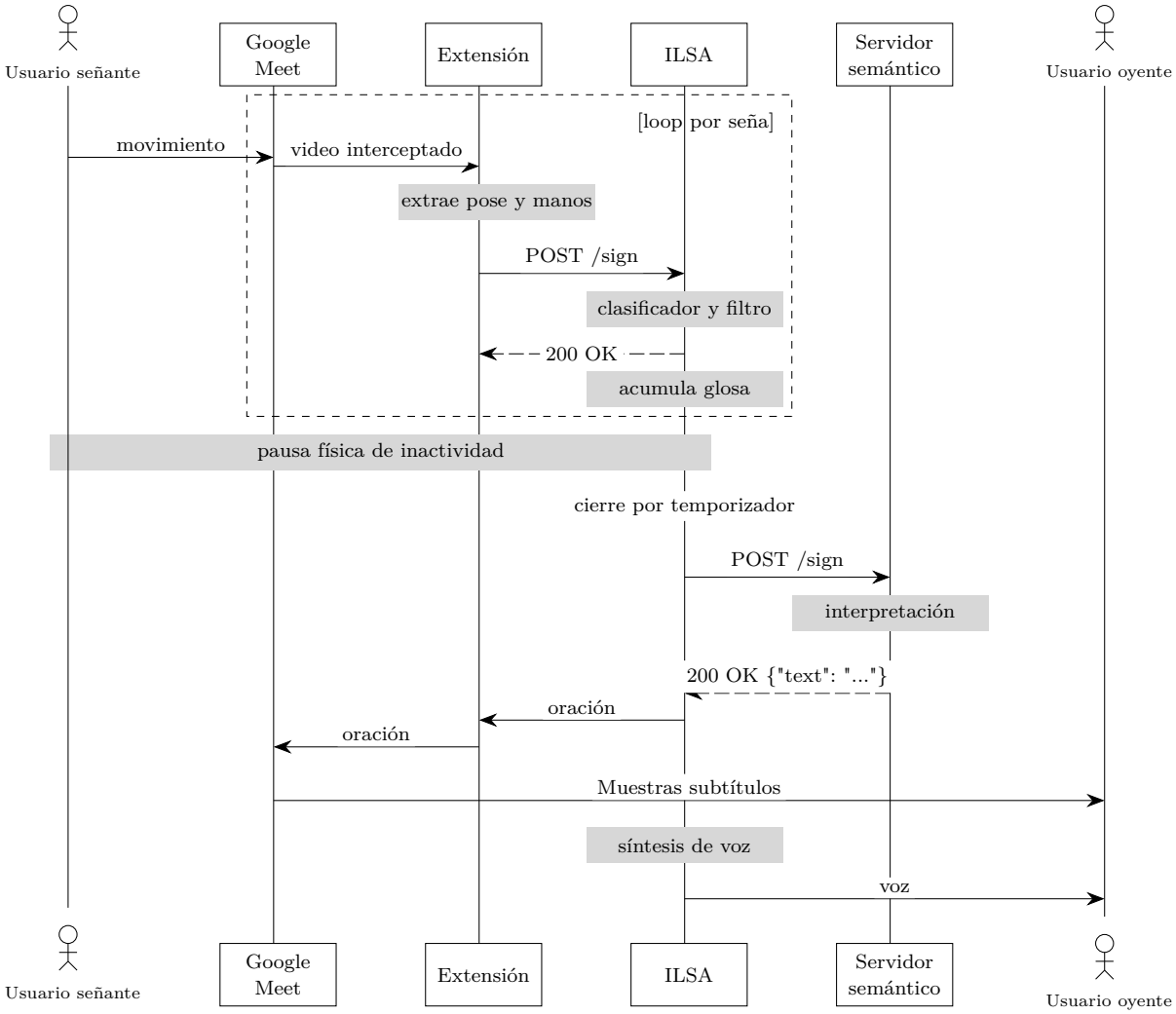


Figura 5.2: Diagrama de secuencia del flujo de traducción de Lengua de Señas a voz y subtítulos.

5.3.1. Procesamiento geométrico de coordenadas y segmentación temporal

Los clips cinemáticos registran la secuencia de puntos clave extraídos mediante la ejecución previa e independiente de *MediaPipe Holistic* sobre el video de un informante calificado. Las coordenadas planares (x, y) se representan en el espacio discreto de píxeles de la cámara de captura ($x \in [0, W]$, $y \in [0, H]$), preservando las proporciones nativas. La componente de profundidad z posee carácter relativo con origen en la muñeca, actuando como métrica para resolver oclusiones mutuas. Por consiguiente, la proyección gráfica reproyecta x e y directamente sobre el plano bidimensional.

El archivo binario `.pose` conserva las tres fases fonológicas estándar (preparación, núcleo, retracción). Para aislar el tramo con valor morfosintáctico distintivo sin alterar el archivo original, el sistema registra los marcadores temporales de inicio (t_{inicio}) y fin (t_{fin}) del núcleo en un índice de anotaciones externo. Durante la renderización, la lectura consulta este índice para aislar el segmento relevante.

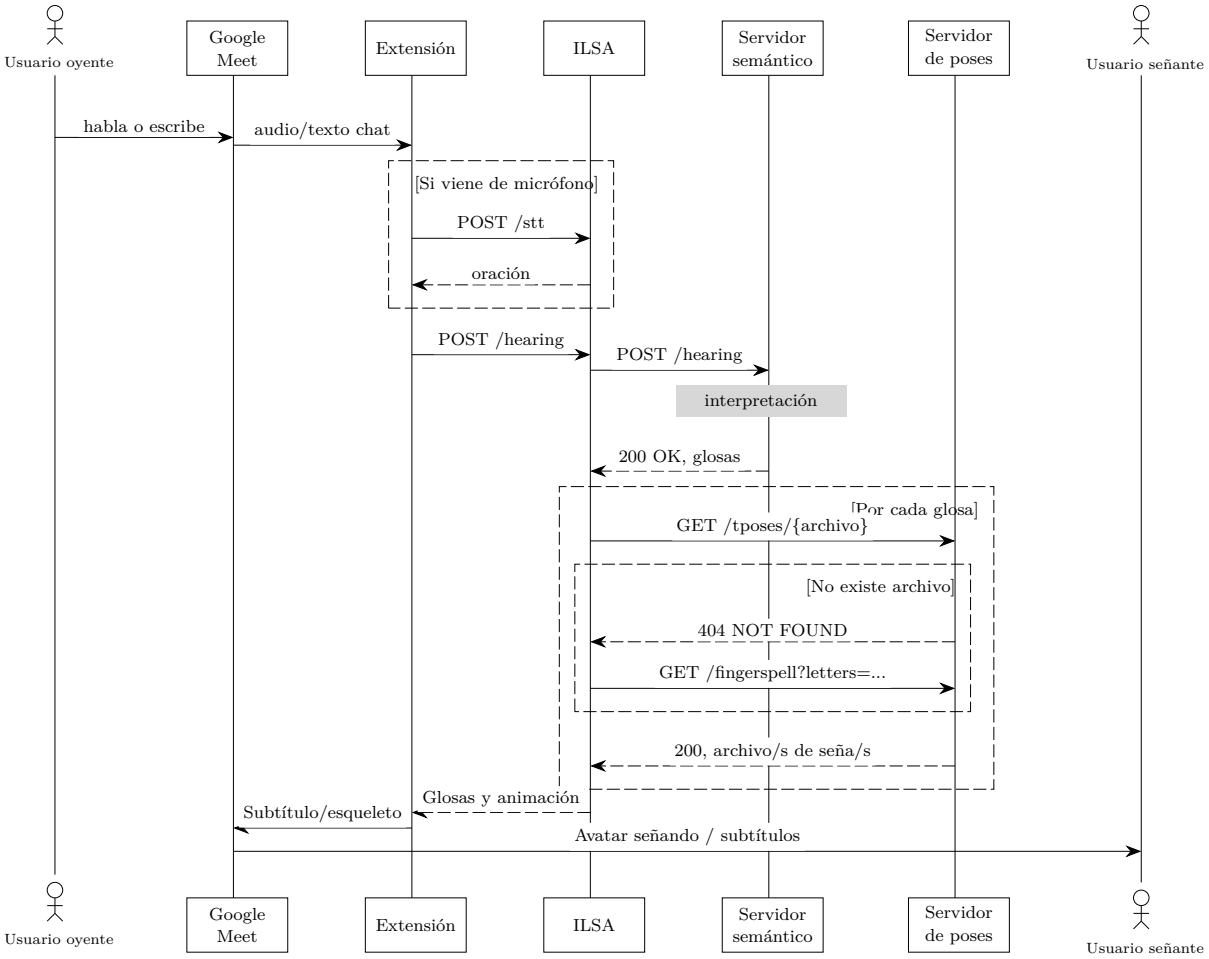


Figura 5.3: Diagrama de secuencia del flujo de interpretación de voz/texto a señas para el usuario señante.

5.3.2. Preparación, acondicionamiento y ensamblado cinemático

Los archivos .pose cargados en el servidor atraviesan una etapa de acondicionamiento estructurada:

1. **Normalización temporal:** Conversión de los clips a una cadencia común de 30 fotogramas por segundo.
2. **Aislamiento del núcleo:** Recorte del tramo delimitado por t_{inicio} y t_{fin}.
3. **Interpolación de faltantes:** Relleno de ausencias puntuales mediante interpolación entre fotogramas válidos adyacentes.
4. **Ajuste de extremidad inactiva:** Reposicionamiento de la mano no participante hacia el costado del torso.
5. **Filtro de suavizado:** Aplicación de un promedio exponencial para atenuar la oscilación entre fotogramas.
6. **Ajuste de escala:** Normalización del tamaño corporal en función de una distancia inter-hombros fija.

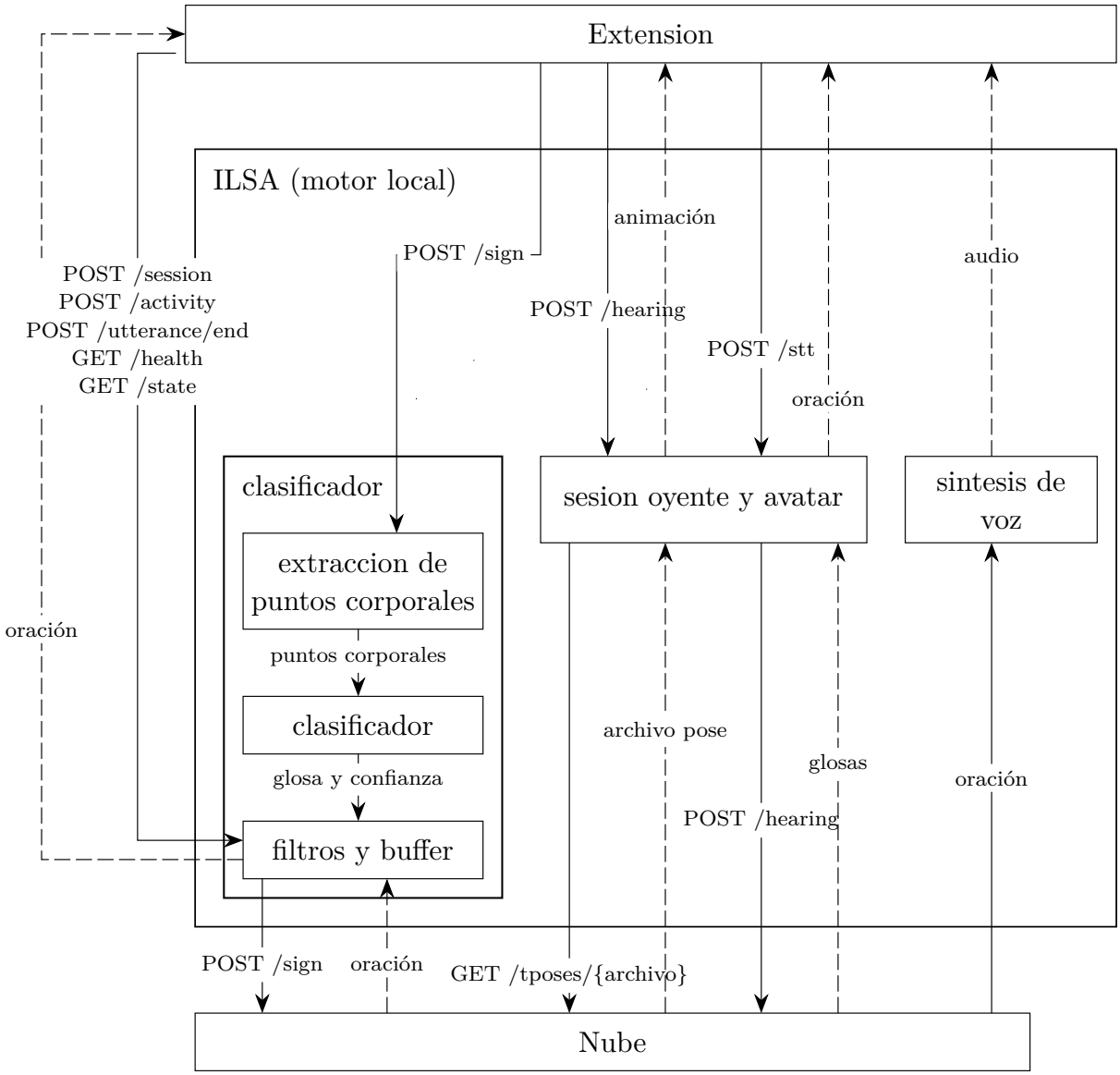


Figura 5.4: Arquitectura detallada del motor local ILSA y su interacción con los servicios en la nube.

Antes de la reproducción de los clips, para evitar discontinuidades espaciales abruptas al empalmar el núcleo del gesto k con el núcleo $k+1$, el sistema interpola un *puente sintético* de fotogramas de transición:

$$S(x) = 3x^2 - 2x^3 \quad \text{para } x \in [0,1] \tag{5.1}$$

Este puente ejecuta una traslación suave sobre las coordenadas planares. Se explicita que este procedimiento constituye un ajuste gráfico e interpolatorio, diferendo del fenómeno biomecánico natural de la coarticulación gestual. El Cuadro 5.4 sintetiza las previsiones de diseño ante contingencias en la representación.

Cuadro 5.4: Previsiones del diseño frente a fallos de estimación o léxico.

Situación	Previsión
Puntos nulos puntuales en el estimador.	Interpolación lineal a partir de fotogramas válidos vecinos.
Inexistencia de archivo para una glosa.	Activación del subsistema de deletreo manual dactilológico.
Secuencias extensas ($>$ 24 glosas o $>$ 1200 cuadros).	Truncado de la secuencia en el límite máximo tolerable.

5.3.3. Estrategia de contingencia léxica

Ante palabras ausentes en el vocabulario grabado (nombres propios o términos técnicos), el sistema aplica un mecanismo de descomposición dactilológica. La palabra se segmenta en sus letras constituyentes y se sintetiza mediante la secuencia correspondiente de caracteres del alfabeto dactilológico, consultando el endpoint específico `/fingerspell`.

5.4. Diseño de la extensión de videollamada

La integración con la plataforma Google Meet se resuelve mediante una extensión de navegador que intercepta la señal de video y gestiona la superposición de capas gráficas. La Figura 5.5 detalla la distribución funcional de sus componentes.

5.5. Patrones de diseño de software aplicados

El diseño incorpora patrones de software consolidados para organizar las responsabilidades del sistema:

- **Estrategia (*Strategy*):** Aplicado en el módulo de filtrado de repeticiones (`RepeatGate`). Define políticas diferenciadas según la clase léxica (dígitos admiten repetición continua, letras aceptan hasta dos instancias consecutivas y vocabulario general descarta duplicados inmediatos).
- **Fachada (*Facade*):** Aplicado en la API HTTP del motor ILSA local. Expone una interfaz unificada que oculta la complejidad interna de extracción, clasificación y comunicación con la nube.
- **Adaptador (*Adapter*):** Implementado en la extensión web y en los conectores de red para transformar los formatos nativos del navegador a los contratos JSON requeridos por los servicios REST.
- **Estado (*State*):** Gestiona la modalidad operativa de la sesión (Señante u Oyente), alterando dinámicamente el comportamiento de los endpoints sin requerir instancias de servicio separadas.

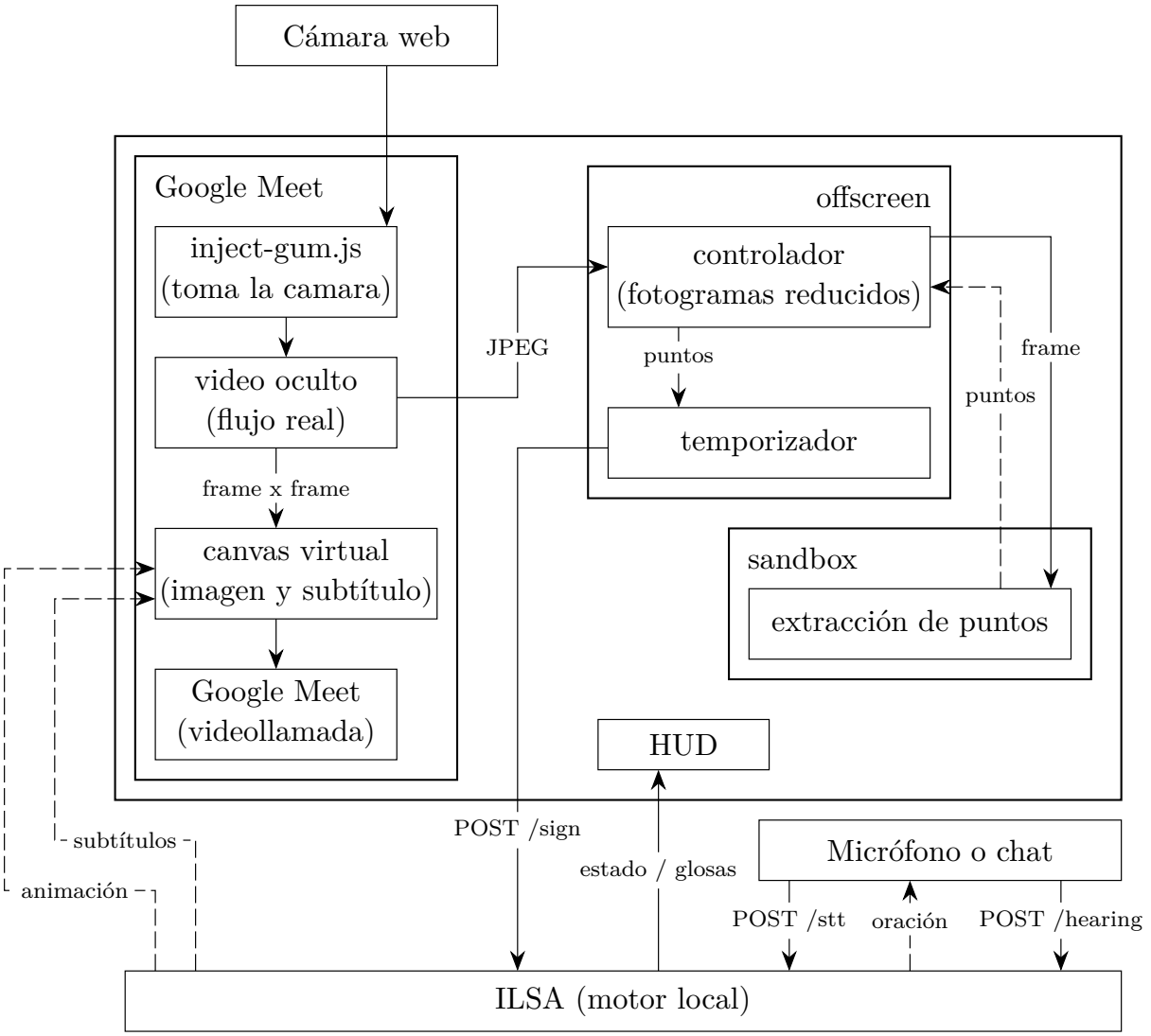


Figura 5.5: Diagrama de arquitectura del flujo de video, procesamiento offscreen e integración con el motor ILSA.

5.6. Diseño conceptual de la arquitectura escalable de producción en la nube

Las pruebas de este trabajo usan una instancia de Cloud Run por servicio. Esta sección no describe esa instancia. Describe la proyección para varias videollamadas a la vez, en coincidencia con la restricción RC-02.

El diagrama, en hoja horizontal, está en el Apéndice C (Figura C.1).

La proyección tiene dos servicios detrás de un mismo ingreso. El traductor genera texto. El servicio de archivos entrega un clip de seña. No comparten réplica ni cola.

5.6.1. Ingreso

Las computadoras cliente salen por HTTPS. El ingreso resuelve el nombre, presenta el certificado TLS y filtra tráfico elemental. Detrás opera un balanceador.

- `POST /sign` y `POST /hearing` van al traductor. El criterio es el menor número de conexiones activas, para no cargar una réplica que ya genera un enunciado largo.
- `GET /tposes/{glosa}.pose`, `GET /fingerspell` y la salud de ese servicio van a las réplicas de archivos.

5.6.2. Traductor

El traductor viaja en una imagen de contenedor. Cada réplica monta los binarios GGUF Q4_K_M desde un volumen de solo lectura, así un arranque nuevo no vuelve a bajar los modelos. Un escalador horizontal mira la cola y el tiempo de generación. Si los turnos simultáneos superan a las réplicas activas, el orquestador abre contenedores nuevos. Por la cuantización a 4 bits, cada réplica cabe en menos de 2 GB de RAM y corre en procesador, sin placa de video.

5.6.3. Archivos de seña

Una petición de pose es una lectura, no una generación. El contrato es el de la Sección 5.2.3. Las réplicas no guardan estado y leen la misma carpeta. El escalador de este grupo mira la cantidad de lecturas, no la cola del traductor. La ruta de salud queda aparte de la ruta del archivo. La caché del cliente sigue como primer recurso.

5.6.4. Monitoreo

Un recolector de métricas observa la latencia por oración, la memoria de cada contenedor del traductor, los errores de red y los errores de lectura del servicio de archivos. Esas series ajustan los dos escaladores.

5.6.5. Qué queda fuera de las pruebas

El prototipo no crea el balanceador, ni los escaladores, ni el volumen compartido. ILSA consulta una instancia del traductor, con los dos canales en un solo proceso, y una instancia del servicio de archivos. Esas dos publicaciones se describen en la Sección 6.6. La carga concurrente y el auto-escalado no entran en la validación.

Capítulo 6

Implementación y Desarrollo

6.1. Estructura del código

El código del prototipo no vive en una única rama del repositorio. Cada pieza del sistema tiene su rama, con una responsabilidad acotada, de modo que el cliente de la videollamada, los entrenamientos, el servicio que ejecuta los traductores en la nube y el servicio de poses se pueden leer y ampliar por separado. Esta sección indica para qué existe cada rama. El detalle de directorios, módulos y convenciones de nombres queda en el `README` de cada uno.

6.1.1. Repositorio del clasificador

6.1.2. Cliente de la videollamada

Este repositorio produce lo que la persona instala para usar el prototipo en una llamada. La extensión se obtendria desde la tienda del navegador y, al instalarse, abre una página desde la cual se descarga el motor local.

El código que arma esas dos piezas se reparte en tres zonas. `extension/` es la extensión: los scripts de la página de la llamada, el dibujo del avatar y la página de bienvenida que entrega el motor. `src/` es el motor local, con `backend/` para la ventana y la API, `classifier/` y `core/` para la seña y la frase, `semantic/` como cliente del traductor remoto, `avatar/` como cliente de las poses y `poseclip/` para componer el clip. `packaging/` empaqueta ese motor en el archivo que descarga la página de bienvenida.

6.1.3. Servicio semántico en la nube

Este repositorio es el servicio que el motor local consulta para traducir. Atiende las dos direcciones, de glosas hacia español y de español hacia glosas, con los binarios que entregan los repositorios de entrenamiento. Su tarea es responder un enunciado ya armado, y no forma parte de lo que se instala en la computadora de quien llama.

El arranque está en `run_semantic_server.py`. El servicio vive en `src/semantic/`: `api.py` expone las rutas, `service.py` separa los dos canales y `engine.py` ejecuta el binario. Los pesos no comparten carpeta. `models/sign/` guarda el canal de glosas hacia español y `models/hearing/` el canal inverso. En cada una hay un binario principal, un respaldo y el `sys_prompt.txt` de esa dirección.

6.1.4. Entrenamiento del canal directo

Este repositorio ajusta y evalúa los modelos que pasan de glosas a una oración en español. El binario que resulta de ese entrenamiento es el que después carga el servicio en la nube para el canal directo.

Todo el código está en `src/semantic/`. `train.py` ajusta un modelo y exporta el binario, `eval_trial.py` lo compara sobre oraciones fuera del corte, y `models.py` es el catálogo: sumar una familia es agregar una entrada ahí. El corpus está en `dataset_glosses.json`, las oraciones de prueba en `dataset_trial.json` y las instrucciones del modelo en `prompts/sys_prompt.txt`. Los pesos y los registros salen a `outputs/`. Cada familia se identifica con un id estable, como `qwen2.5-1.5b` o `llama-3.2-1b`, y ese mismo id nombra su carpeta de salida.

6.1.5. Entrenamiento del canal inverso

Este repositorio ajusta y evalúa los modelos que pasan de una oración en español a una secuencia de glosas. El corpus y el binario son propios de esta dirección, y el servicio en la nube los usa para el canal inverso.

La disposición repite la del canal directo, también bajo `src/semantic/` y también con los módulos en inglés: `train.py` entrena, `eval_trial.py` compara y `config.py` reúne rutas, hiperparámetros y la lista de modelos. El corpus, el archivo de prueba y el prompt ocupan los mismos nombres de archivo que en el otro laboratorio. Los id de modelo también coinciden, de modo que `qwen2.5-1.5b` designa la misma familia en las dos direcciones, con pesos distintos.

6.1.6. Servicio de poses del avatar

Este repositorio publica los archivos de pose que el motor local pide para dibujar el avatar. No entrena modelos ni traduce texto. El cliente de la videollamada solicita esas poses a este servicio.

El código cabe en `app/`: `main.py` arranca el servicio, `config.py` fija rutas y tipos de archivo, y `fingerspell.py` resuelve el alfabeto manual. Las poses están en `poses/`. El nombre del archivo, sin extensión, es la glosa ya normalizada: mayúsculas, sin tildes y en ASCII. Dentro de una palabra la eñe pasa a ene, y la letra eñe, cuando es la glosa entera, se pide como `ENIE`.

6.2. Implementación del subsistema de reconocimiento gestual

6.2.1. Preprocesamiento de datos y pipeline de extracción

El pipeline procesa fotogramas de entrada mediante *MediaPipe Holistic* para estructurar un vector instantáneo de 225 coordenadas (33×3 de pose, 21×3 de mano izquierda y 21×3 de mano derecha). Para corregir discrepancias en la relación de aspecto de las cámaras ($4:3$ versus $16:9$), se aplica un reencuadre con bandas de relleno a una resolución uniforme de 853×480 píxeles previa extracción. Cada vector se centra en el punto medio de los hombros y se normaliza dividiendo por la distancia inter-hombros.

6.2.2. Arquitectura neuronal TinySkeletonClassifier

La clasificación de la secuencia temporal de 16 fotogramas se ejecuta mediante el modelo *TinySkeletonClassifier*, cuya implementación comprende:

1. Dos capas de convolución unidimensional temporal (núcleo de tamaño 3, 128 canales, normalización por lotes, activación ReLU y regularización Dropout de 0,2).

2. Un módulo de codificación posicional sinusoidal sumado a la representación de 128 dimensiones.

3. Un codificador Transformer de 2 capas con 4 cabezas de atención.

4. Una capa de agregación por atención (*Attention Pooling*) que asigna un escalar de relevancia a_t a cada fotograma t, reduciendo la secuencia temporal a un vector único de 128 dimensiones:
$$\mathbf{v} = \sum_{t=1}^{T} a_t \mathbf{h}_t \tag{6.1}$$

5. Una capa densa lineal final hacia las 100 clases del vocabulario gestual.

6.3. Implementación del subsistema de traducción semántico bidireccional

6.3.1. Selección y especificación de la suite de modelos de lenguaje

Para la traducción semántica bidireccional se adaptaron modelos de lenguaje pequeños de entre 0,5B y 3B parámetros:

- **Familia Qwen2.5:** Variantes de instrucciones de 0,5B, 1,5B y 3B parámetros.
- **Familia Llama 3.2:** Variante de instrucciones de 1B parámetros.
- **Familia SmolLM2:** Variante de instrucciones de 1,7B parámetros.

En el servidor, cada canal arranca con Qwen2.5 de 1,5B y, si ese archivo no está disponible, con Llama 3.2 de 1B. El contrato de esas rutas está en la Sección 5.2.2.

6.3.2. Régimen de entrenamiento y ajuste LoRA

El canal directo y el canal inverso comparten el mismo procedimiento de entrenamiento. El corpus se divide en un 85 % para entrenamiento y un 15 % para evaluación. Esta separación utiliza una semilla fija para asegurar que cualquier prueba sea reproducible en las mismas condiciones. Cada ejemplo del conjunto de datos representa una interacción de tres partes: las instrucciones iniciales del sistema (*prompt*), el mensaje del usuario y la respuesta esperada. La ventana de procesamiento admite hasta 2048 tokens.

Para adaptar el modelo a la lengua de señas sin modificar sus miles de millones de parámetros originales, se utiliza la técnica LoRA. Se eligió un rango de adaptación $r = 16$ y un factor de escala $\alpha = 32$. La relación $\alpha/r = 2$. Estos valores responden a la configuración recomendada para modelos de este volumen.

Las matrices que reciben entrenamiento son las cuatro capas de proyección de atención y las tres capas de la red de avance. En cambio, la tabla de tokens de entrada y la capa de salida permanecen fijas. Esta elección se debe a que el vocabulario base del modelo no cambia; la tarea solo necesita reorganizar y conectar palabras que la red ya conoce.

El tamaño de lote efectivo se fijó en 16 ejemplos por cada paso del optimizador. Dado que el modelo de 3B parámetros consume más memoria, su micro-lote se redujo a 2 ejemplos y los gradientes se acumulan durante 8 pasos. Los modelos más pequeños utilizan micro-lotes de 4 ejemplos y acumulan durante 4 pasos. De esta forma, el paso matemático del optimizador es idéntico entre todas las opciones y los resultados son comparables.

El *learning rate* es de 3×10^{-4} para el modelo de 3B y de 5×10^{-4} para las versiones más pequeñas. Se asignó un valor menor al modelo grande porque un ajuste brusco en una red con más capas provoca inestabilidad en el aprendizaje. El optimizador elegido es AdamW cuantizado a 8 bits. Esta elección reduce el uso de memoria RAM en la GPU, ya que el estado del optimizador se guarda en 8 bits y convive sin problemas junto al modelo base de 4 bits. El decaimiento de pesos de 0,01 evita que los adaptadores crezcan demasiado y memoricen los datos. Se incluyen 10 pasos de calentamiento para que la tasa de aprendizaje suba de forma gradual desde cero, lo que evita desajustes iniciales bruscos en las matrices de LoRA que arrancan con valores al azar. Luego, un planificador de tipo coseno reduce de forma suave la tasa de aprendizaje hacia el cierre del entrenamiento para afinar los ajustes finales.

Para acelerar el entrenamiento y no perder capacidad de cómputo, la función de pérdida no evalúa todo el diálogo. Una máscara calcula el error únicamente sobre la respuesta deseada, mientras que el *prompt* del sistema y la entrada del usuario quedan fuera de la actualización de pesos. De este modo, la red concentra todo su aprendizaje en la fluidez de la traducción.

Por último, antes de ingresar al entrenamiento, los datos reciben una limpieza específica. Las letras sueltas y consecutivas del dataset se unen en un solo token con mayúscula inicial, y los dígitos sueltos forman un número consolidado. Tres letras idénticas seguidas se reducen a dos para corregir errores de tipeo o captura. Las secuencias de palabras sin separación previa, como *Mar del Plata*, se agrupan en un único token *Mardelplata* para que la falta de espacios no confunda al modelo durante el aprendizaje.

Procedimiento de evaluación y post-entrenamiento

Al cierre del entrenamiento, el 15 % del conjunto de datos reservado se evalúa sobre el binario en formato GGUF cuantizado a 4 bits. Esta prueba mide el rendimiento real sobre el servidor de inferencia en producción, y no el del modelo en precisión flotante original.La evaluación configura los siguientes parámetros generales de generación y medición:

- **Determinismo:** Una temperatura de valor 0 anula la aleatoriedad en la generación de texto y fuerza al modelo a elegir siempre el token de mayor probabilidad matemática. De esta forma, las respuestas resultan estrictamente reproducibles.

- **Control de repetición y parada:** Se aplica una penalización de repetición de 1,05 para desincentivar que el modelo entre en bucles de palabras o glosas idénticas durante la inferencia. El proceso se detiene ante la aparición de un salto de línea o de la marca de fin de turno, lo que evita el consumo innecesario de cómputo.

- **Múltiples referencias:** Cuando existen varias salidas válidas para una misma entrada, el sistema las agrupa a todas como referencias aceptadas para no penalizar

variantes correctas.

- **Métricas de rendimiento:** La prueba registra el acierto exacto, el acierto tras normalizar puntuación y mayúsculas, y cinco métricas de texto: BLEU, ROUGE-L, METEOR, `BERT Score` y `Comet Score`. BLEU cuenta n-gramas compartidos con la referencia. ROUGE-L cuenta la subsecuencia común más larga. METEOR alinea palabras, raíces y sinónimos. `BERT Score` mide la cercanía de los vectores de los tokens. `Comet Score` estima la calidad de la traducción a partir de la entrada, la salida y la referencia. Van juntas porque el solapamiento de palabras no alcanza para decidir si se conservó el sentido. El detalle de cada una está en la Sección 3.3. La latencia se registra aparte, como tiempo de respuesta.

6.3.3. Canal directo (LSA → Español)

El par de este canal es glosas hacia una oración en español rioplatense. La entrada del usuario antepone «Glosas:» a la secuencia ya compacta. La consigna del ajuste pide una sola oración, sin notas ni datos ajenos a la entrada:

- El español es rioplatense y usa voseo. La oración incorpora artículos, preposiciones y concordancia verbal. La traducción no es palabra por palabra.
- Los pronombres de entrada son `YO`, `VOS`, `EL`, `NOSOTROS` y `ELLOS`. Sin una marca de género al lado, el sujeto queda tácito.
- La letra `A` inmediata al sustantivo o al pronombre marca femenino, y la letra `O` marca masculino. Un parentesco sin esa letra se considera masculino.
- `MIO` y `TUYO` expresan posesivos del español.
- `PASADO` o `AYER` exigen pretérito. `FUTURO` exige tiempo futuro. Sin marca temporal, el verbo queda en presente.
- `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` y `CUANTOS` producen una pregunta directa, con signos de apertura y de cierre.
- `NO` junto al concepto o al verbo produce una negación.
- Un token en capitalización de nombre, o una cifra, se copia tal cual. La consigna prohíbe sustituirlo o alterarlo.
- La entrada sigue el orden *Topic-Comment* (tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y pregunta). La oración de salida reordena esa secuencia según la sintaxis del español.

Para la evaluación post-entrenamiento de este canal, se establece un límite máximo de 24 tokens para la salida. Esta restricción responde a que las oraciones traducidas del vocabulario base son breves, y frena la generación de texto infinito en caso de un fallo en la red.

6.3.4. Canal inverso (Español → LSA)

El par invierte al canal directo: la entrada es la oración y la respuesta son las glosas. La entrada antepone «Español:». La máscara deja el error sobre las glosas.Pide una sola secuencia, separada por espacios, sin notas, sin el prefijo «Glosas:» y sin artículos ni preposiciones del español:

- Las glosas léxicas van en mayúsculas sostenidas. Un nombre, un apellido o una calle va en capitalización de nombre, como `Patricia` o `Moreno`, y nunca letra por letra. Una cifra, como un documento o una cantidad, va en un solo token numérico.
- El orden es tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y, al cierre, la partícula interrogativa o la negación.
- Los pronombres se explicitan aunque el español los omita. Tengo un cuchillo.” → `YO CUCHILLO TENER`. El mapeo es yo → `YO`, vos o tú → `VOS`, él → `EL O`, ella → `EL A`, nosotros → `NOSOTROS`, nosotras → `NOSOTROS A`, ellos → `ELLOS O` y ellas → `ELLOS A`.
- El género del parentesco usa el sustantivo en masculino más `O` o `A`: hermana → `HERMANO A`, hija → `HIJO A`, esposa → `ESPOSO A`. Un sustantivo de persona sin marca de género queda en masculino y suma `O` si el español lo explicita.
- `MIO` y `TUYO` cubren los posesivos. Me llamo X” o Mi nombre es X” → `YO NOMBRE` más el nombre.
- El pretérito o el copretérito colocan `PASADO` al inicio. El futuro, o la perífrasis voy a”, coloca `FUTURO`. El presente sin marca no agrega token temporal.
- Las preguntas de dónde, quién, cómo, qué, cuándo y cuántos dejan `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` o `CUANTOS` al final. La negación deja `NO` al final, junto al verbo.

Mediante estas reglas, el modelo aprende a emitir nombres propios y cifras ya empaquetados para la rutina de dactilología del avatar, en lugar de generar cadenas incoherentes de letras.En la evaluación post-entrenamiento de este canal, el límite de generación se fija en 48 tokens. Este margen es más amplio que el del canal directo porque las oraciones en español que contienen nombres propios o cifras largas producen secuencias extensas de caracteres dactilológicos en la salida.

6.3.5. Cuantización GGUF Q4_K_M y estimación de memoria

Los modelos ajustados se fusionaron y convirtieron a binarios GGUF con cuantización `Q4_K_M`. La memoria RAM requerida durante la inferencia se calcula mediante la relación:

$$\text{RAM} \approx \mathbf{W} + KV_{\text{fp16}}(ctx = 2048) + \text{Buffer} \tag{6.2}$$

donde $\mathbf{W}$ representa el tamaño de los tensores cuantizados ($\sim 0{,}5$ bytes por parámetro), KV_{fp16} la caché de claves y valores para 2048 tokens de contexto, y el buffer reserva espacio para las activaciones intermedias.

Las Figuras 6.1 y 6.2 aplican esa cuenta a los cinco modelos. El término de pesos usa los totales publicados: 0,49 mil millones para Qwen2.5 0,5B, 1,54 para el 1,5B, 3,09 para el 3B [31], 1,23 para Llama 3.2 1B [33] y 1,71 para SmolLM2 1,7B [32]. Antes de

cuantizar, el modelo fusionado queda en 16 bits, es decir 2 bytes por parámetro. Después, la aproximación de la ecuación usa 0,5 bytes por parámetro. La caché no cambia con la cuantización, porque la ecuación la deja en fp16:

$$KV_{\text{fp16}} = 2 \cdot L \cdot n_{\text{kv}} \cdot d \cdot 2048 \cdot 2$$

en bytes. El primer 2 son las matrices de claves y de valores. El último 2 es el tamaño de un valor fp16. L es el número de capas, n_{kv} el de cabezas de clave y valor, y d el ancho de cada cabeza. Esos anchos salen de la configuración publicada: 64, 128 y 128 para los tres Qwen, 64 para Llama y 64 para SmolLM2. Las cabezas de clave y valor son 2, 2, 2, 8 y 32, en el mismo orden. El buffer de la ecuación no tiene un valor fijo en este capítulo, así que las barras no lo suman. La cifra es teórica. No es la memoria que midió el proceso en la máquina de las pruebas.

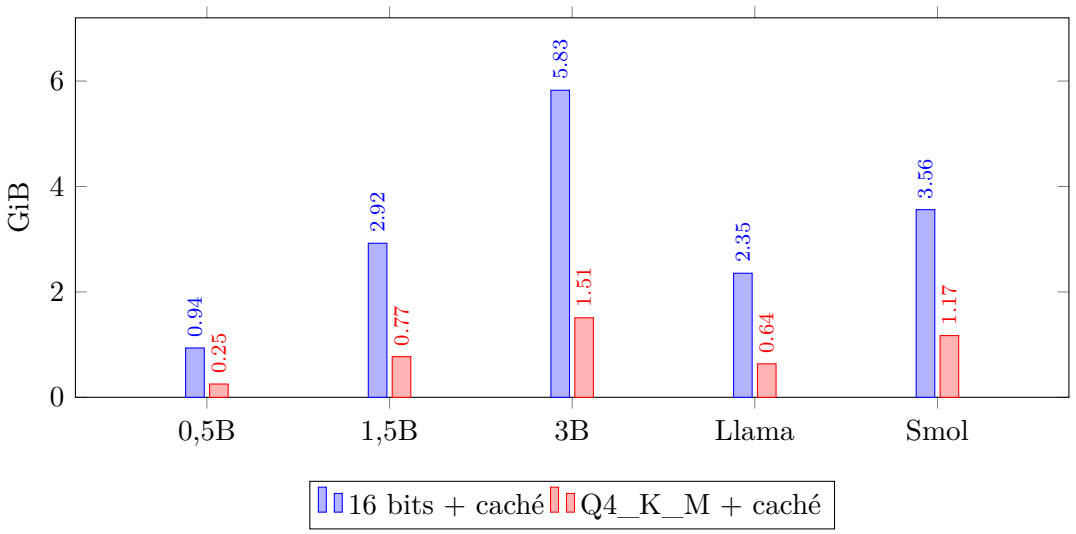


Figura 6.1: RAM teórica de inferencia, $W + KV_{\text{fp16}}(2048)$, en GiB. El orden del eje es Qwen2.5 0,5B, 1,5B y 3B, Llama 3.2 1B y SmolLM2 1,7B. El buffer no entra.

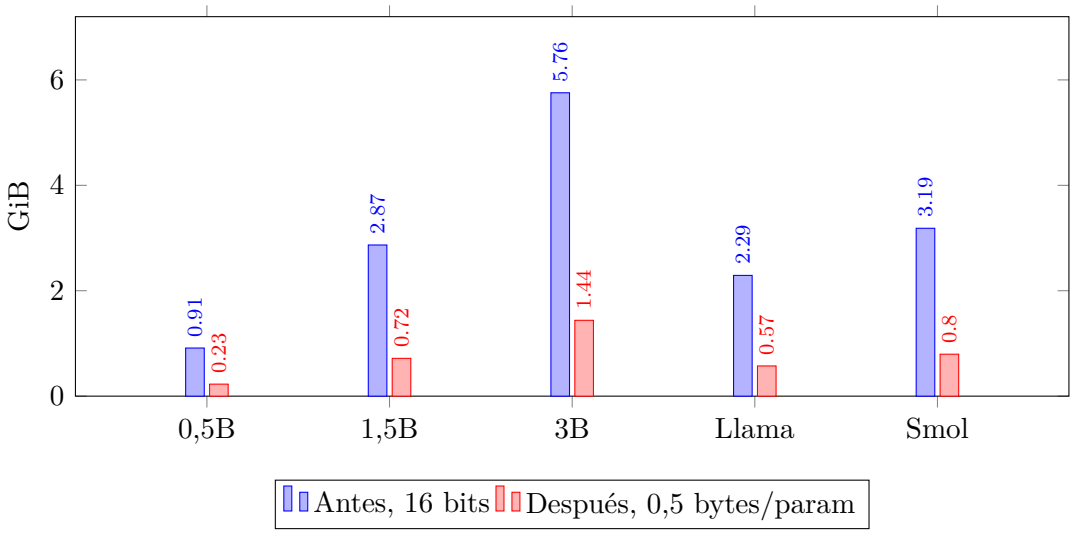


Figura 6.2: Tamaño teórico de los pesos, antes y después de la cuantización, en GiB. Mismo orden de modelos que la Figura 6.1. La barra comprimida usa los 0,5 bytes por parámetro de la ecuación 6.2. El promedio cercano a 5 bits del preset, en el marco teórico, queda un poco por encima de esa barra.

6.4. Integración en Google Meet y arquitectura de la extensión

6.4.1. Secuencia completa de inferencia y comunicación multidireccional

El motor local de la plataforma determina el modo operativo antes de cualquier interacción con la extensión de navegador. A través del panel de control, el usuario configura la lateralidad (diestro o zurdo) y el canal de comunicación (Sordo u Oyente). Las acciones de encendido, reinicio o apagado retornan a la pantalla de carga. La interfaz principal solo habilita sus funciones si el traductor semántico y el servidor de posturas responden correctamente.

El proceso principal escucha peticiones en una dirección de red local. Dado que las políticas de seguridad de Google Chrome identifican esa dirección como una red interna restringida, la página de la llamada no puede comunicarse con ella de forma directa. Por este motivo, el módulo en segundo plano de la extensión opera como intermediario, mientras que el documento secundario sin pantalla y la ventana emergente —al pertenecer al mismo origen de la extensión— entran en contacto directo con el motor local.

Durante la navegación en una llamada activa, el módulo en segundo plano efectúa una consulta inicial sobre el estado del sistema. Si el motor local se encuentra encendido, la extensión inyecta herramientas en dos entornos aislados del navegador:

- **Entorno de soporte:** Gestiona la interfaz flotante, las preferencias del usuario y el envío de solicitudes de red.
- **Entorno de la página principal:** Encargado de la renderización en pantalla y de tomar el control de la transmisión de la cámara web.

El sistema reitera la validación de estado cada dos segundos. Si el traductor no se encuentra listo, la interfaz solicita el reinicio del servicio y detiene el flujo. La modalidad seleccionada determina el canal activo, con la restricción de que el modo Sordo y el modo Oyente nunca se ejecutan en simultáneo.

El ciclo continuo de inferencia, procesamiento y comunicación se articula a través de las siguientes etapas:

1. **Gestión de la señal de captura en cámara única:** La extensión intercepta y sustituye la función nativa de acceso a la cámara. El dispositivo físico realiza una única apertura y deriva su señal de video hacia un elemento oculto dentro de la página. La llamada de la plataforma no recibe este flujo directo; en su lugar, recibe la señal sintética de un lienzo virtual a 24 cuadros por segundo. El transmisor de video de la llamada sustituye su fuente original por la de este lienzo. Al apagar la extensión, la página devuelve la señal nativa a la llamada sin detener abruptamente el hardware. Esta estrategia evita errores de dispositivo ocupado o no disponible, los cuales ocurren cuando la llamada exige el control exclusivo de la cámara física.

2. **Transmisión de fotogramas hacia el clasificador (Modo Sordo):** En la modalidad para personas sordas, la extensión extrae las imágenes de la cámara oculta y las comprime en un formato liviano con resolución y calidad reducidas. El sistema envía fotogramas y aguarda la confirmación antes de despachar el siguiente. El mensaje viaja internamente a través del módulo en segundo plano sin abandonar la computadora ni ingresar de forma directa al motor principal ILSA. Si la cámara de la llamada se cierra, la página emite un aviso para reiniciar el proceso de captura y recorte.

3. **Estimación de puntos articulares en el entorno aislado:** Las restricciones de seguridad del navegador impiden ejecutar la herramienta de visión artificial directamente en el módulo en segundo plano. Por esta razón, el sistema decodifica la imagen comprimida sobre un lienzo, extrae los píxeles y los transfiere hacia un documento aislado y protegido. Con esta información, el sistema estructura un vector cinemático. Si el usuario seleccionó la opción de lateralidad zurda, el servidor activa la inversión automática de ejes y el cruce de manos en el vector, ajustando los datos al modelo original que fue entrenado con muestras de mano derecha. La imagen comprimida no viaja en este envío.

4. **Criterio de corte y segmentación gestual:** El algoritmo de recorte opera dentro de la extensión en modo automático, sin evaluar el movimiento de los píxeles para evitar interferencias si hay personas caminando en el fondo. El inicio de la seña se registra al detectar una mano utilizable en la imagen. El cierre ocurre cuando las manos abandonan el encuadre, con un margen de tolerancia frente a eventuales pérdidas temporales de detección. El sistema no interrumpe una seña estática por falta de movimiento. El espacio de memoria guarda hasta 60 cuadros y realiza un muestreo de 16 posiciones antes de la transmisión.

5. **Clasificación local y filtrado mediante compuertas:** Cada seña segmentada se envía mediante un paquete de datos que contiene únicamente el vector de coordenadas. El clasificador procesa la información dentro del motor local. Si el nivel de confianza es inferior al umbral mínimo del 75 %, la predicción se descarta y la interfaz la presenta en color coral con la etiqueta «Poca confianza». Si alcanza el umbral pero la

compuerta de validación frena la entrada, la interfaz la despliega en tono gris como «No se agregó». Si la seña aprueba las validaciones, ingresa a la frase, el panel la destaca en cian con la indicación «En el mensaje» y el listado de la frase se actualiza.

6. **Finalización de enunciado y traducción semántica:** Tras un intervalo de 4 segundos sin actividad manual en la cámara, el sistema transmite la orden de cierre de frase. El motor local no envía la totalidad de los datos a servicios en la nube: las secuencias de deletreo, las series numéricas o las señas aisladas se resuelven de forma local. Las cadenas complejas se envían como texto de señas hacia el servidor semántico remoto. Si el módulo de traducción no emite respuesta, el motor conserva las señas aceptadas en formato de texto original sin reestructurar la oración.

7. **Emisión de salidas en el Canal Directo (Modo Sordo):** El sistema recibe la traducción en español y la publica en la página de la llamada. El módulo de video graba este texto sobre una franja oscura en la zona inferior del lienzo virtual, el cual conforma la imagen que los demás participantes observan. La capa visual del panel mantiene el texto visible durante 8 segundos. En la modalidad con salida por Subtítulo no se genera audio. En las configuraciones con Audio o Salida Doble, la página solicita una locución en español a la interfaz de voz nativa del navegador. Si una frase traducida es idéntica a la inmediata anterior, el sistema no la vuelve a reproducir.

8. **Procesamiento de entrada por voz en el Canal Inverso (Modo Oyente):** En la modalidad para personas oyentes el sistema captura el audio a través de un módulo de procesamiento sonoro. La extensión envía el bloque de audio comprimido al motor local, donde un modelo de reconocimiento de voz realiza la transcripción de forma totalmente privada y local. El modelo opera exclusivamente en idioma español. Si la conversión no produce texto, la interfaz solicita repetir la locución o recurrir al canal de texto.

9. **Procesamiento de entrada por texto en el Canal Inverso (Modo Oyente):** El sistema captura el contenido escrito sin pasar por el módulo de voz. La entrada de voz y la de texto pueden permanecer activas de forma concurrente; ambas representan distintas vías de ingreso para el mismo caso de uso y no constituyen un modo separado.

10. **Generación de glosas y renderización de animación sintética:** La cadena de texto en español ingresa al motor local. Las palabras aisladas se procesan como señas directas sin consultar al traductor remoto, mientras que las oraciones compuestas se envían al servicio en la nube y retornan estructuradas como señas en el orden gramatical del sistema. El motor local solicita los archivos de animación cinemática correspondientes a cada seña, con un límite de hasta 24 elementos por enunciado. En ausencia del archivo para una seña específica, el sistema solicita los archivos de cada letra para realizar un deletreo dactilológico. Si el servidor de animaciones no responde, las señas se conservan en texto y el bloque visual permanece vacío. La extensión recibe los datos de movimiento ya unificados por el servidor. En la superficie del lienzo, la cámara del usuario ocupa la mitad izquierda y el modelo de esqueleto ocupa la mitad derecha. La llegada de una nueva animación no interrumpe la secuencia en ejecución, y la cola de reproducción admite hasta 20 bloques animados, descartando el más antiguo al colmar su capacidad. Las señas procesadas también se graban sobre

la franja inferior del mismo video para que el interlocutor las visualice directamente en la llamada.

6.4.2. Motor local ILSA (FastAPI)

El servidor local se implementó en Python mediante el framework asíncrono FastAPI y se ejecuta sobre un servidor Uvicorn. El servicio se enlaza exclusivamente a la interfaz de bucle local (`127.0.0.1:8765`) para denegar conexiones externas no autorizadas.

Para cumplir con las políticas de seguridad del navegador, el middleware del servidor implementa reglas CORS que restringen los orígenes admitidos a `https://meet.google.com` y al identificador de la extensión (`chrome-extension://*`). Asimismo, responde con la cabecera de acceso a red privada (`Access-Control-Allow-Private-Network: true`) requerida cuando una página HTTPS realiza peticiones hacia el dominio local.

La emisión de voz sintética se desacopló mediante un hilo de ejecución secundario que consume una cola secuencial de mensajes, evitando bloqueos en la atención de las solicitudes HTTP de la API. El motor local se empaquetó como un ejecutable independiente de Windows (`ILSA.exe`) utilizando PyInstaller, e incluye una interfaz gráfica desarrollada en Tkinter para permitir la selección de modo (Sordo u Oyente), la configuración de la mano dominante y el control de encendido del servicio.

6.5. Implementación del avatar

6.5.1. Vía descartada: retarget hacia un avatar VRM

Antes del diccionario de poses se probó un personaje VRM, con malla y huesos de longitud fija, animado por rotaciones. El fundamento del ensayo es el *retarget*: el paso de posiciones de *landmarks* a rotaciones de otro esqueleto. MediaPipe Holistic entrega posiciones, y el avatar exige rotaciones sobre su propio esqueleto. El visor se armó con Three.js. Para la mano se probó Kalidokit, una biblioteca de JavaScript para ese *retarget*. Toma los *landmarks* de MediaPipe y calcula rotaciones de Euler del esqueleto y pesos de *blendshape* del rostro. En la mano resume cada dedo en un único ángulo de rizo. También se probó una cinemática inversa que lleva el extremo del hueso hasta el punto medido.

La prueba no se retuvo. El desarrollo no contaba con práctica suficiente de animación tridimensional, y las bibliotecas de *retarget* no sostuvieron de forma estable el paso de la pose al esqueleto VRM. El emparejamiento fallaba con frecuencia porque la pose pierde la profundidad en el eje Z: la coordenada de la mano queda relativa a la muñeca y en una escala distinta del desplazamiento en el plano, de modo que el hueso no recibe una distancia utilizable. Si se respeta la longitud del hueso, la yema no cae sobre el punto medido. Si la cinemática inversa alarga el miembro hasta ese punto, la mano pierde la configuración y, en el alfabeto manual, la letra cambia. Kalidokit, además, confunde configuraciones próximas.

El diccionario de `.pose` evita ese paso: dibuja los puntos medidos y no los convierte en rotaciones de otro cuerpo. Si más adelante se retoma el avatar tridimensional, una línea de mejora es filmar cada seña desde tres ángulos, para reconstruir la profundidad que una sola toma frontal no conserva.

6.5.2. Decisiones técnicas de la reproducción

La Sección 5.3 fija qué se muestra y por qué. Aquí quedan los valores con los que esas reglas se programaron, y cuál de los dos programas hace cada paso. La preparación del clip y el empalme entre glosas corren en ILSA, una vez que los archivos ya llegaron del servicio de poses. La extensión de la videollamada no rehace ese empalme. Recibe la lista de fotogramas ya armada y la dibuja en la mitad derecha del lienzo. El navegador, entonces, no vuelve a calcular el puente: elige, según el tiempo transcurrido, cuál de esos fotogramas le toca mostrar.

Valores del compositor. Antes del recorte, todos los clips se llevan a 30 fotogramas por segundo. Con esa tasa común, una espera de seis fotogramas dura lo mismo en cualquier seña. Si se dejara cada video a la cadencia con la que fue filmado, seis fotogramas de un clip a 24 por segundo durarían más que seis fotogramas de un clip a 30, y el empalme perdería la medida del tiempo.

El suavizado es un promedio exponencial. Si x_t es la posición recién medida y p_{t-1} es la pose ya aceptada en el fotograma anterior, la pose que se conserva es

$$p_t = \alpha\, x_t + (1 - \alpha)\, p_{t-1}, \tag{6.3}$$

con $\alpha = 0{,}50$. La medición nueva y la pose anterior pesan lo mismo. Ese peso es único para todo el diccionario.

El recuadro común mide 720×780 píxeles. El cuerpo se escala para que el ancho de hombros quede en 250 píxeles. Así, dos tomas de distinto encuadre ocupan el mismo espacio cuando se pegan una detrás de la otra.

El puente dura entre 4 y 8 fotogramas. Si las muñecas del final de una seña y del inicio de la siguiente ya están cerca, se usan 4. Si están lejos, se llega a 8, para que la mano no salte. Un punto cuya confianza queda por debajo de 0,15 sale en la lista como ausente, y el trazo del lienzo no lo une con el punto anterior.

La entrada, desde la postura de espera hasta el primer fotograma de la seña, y la salida, de vuelta a esa espera, duran cerca de seis fotogramas de la tasa de 30. A esa cadencia, cada uno de esos tramos ocupa cerca de un quinto de segundo. La secuencia se corta en el techo fijado en el diseño: 24 glosas y 1200 fotogramas.

Postura de espera. No hay un archivo aparte llamado reposo. El cuerpo y las manos de la espera se copian de los primeros fotogramas de una seña de referencia, el tramo en el que la persona todavía no empezó el gesto, y se escalan al mismo ancho de hombros del lienzo. El antebrazo queda casi horizontal, levemente hacia abajo, y la mano sigue esa línea.

Reflejo de la miniatura local. La videollamada muestra la miniatura de quien usa el sistema como un reflejo. La extensión deshace ese reflejo solo en ese video local. El esqueleto que viaja hacia la otra persona no se invierte. Las dos vistas quedan iguales: la cámara en la mitad izquierda y el esqueleto, sin darlo vuelta, en la mitad derecha.

6.6. Instancia de validación del prototipo mediante microservicios en la nube

Para la validación funcional del prototipo en un entorno de producción representativo, se completó el desacoplamiento del backend semántico y del servidor de poses del avatar hacia la infraestructura serverless de Google Cloud Run.

La arquitectura se dividió en dos microservicios independientes, empaquetados en contenedores Docker:

- **Servidor Semántico:** Este microservicio gestiona los modelos cuantizados a 4 bits (GGUF `Q4_K_M`) para la traducción directa e inversa. Se desplegó en la región *us-central1* con una asignación de 8 GiB de memoria RAM y 4 vCPU.
- **Servidor de Poses y Activos del Avatar:** Este microservicio empaqueta y entrega los archivos de configuración, coordenadas de pose (*T-Poses*) y recursos de animación (600 MB de archivos estáticos en formatos `.json`, `.gltf` y `.bin`). Se desplegó en la región *southamerica-east1* con una asignación de 512 MiB de memoria RAM y 1 vCPU.

El proceso de despliegue e integración en la nube se llevó a cabo bajo los siguientes criterios técnicos:

1. **Empaquetamiento e integración continua (CI/CD):** Mediante la herramienta `gcloud builds`, el código fuente y las dependencias de cada rama se enviaron a Google Cloud Build para construir las imágenes de contenedor, las cuales quedaron almacenadas en Google Artifact Registry (`ilsa-repo`).
2. **Gestión dinámica de puertos y comprobación de estado:** A fin de evitar fallos de inicialización (*health-check timeouts*), cada contenedor se configuró para adaptar su punto de entrada a la variable de entorno `$PORT` inyectada de forma dinámica por la plataforma Cloud Run, habilitando la escucha en la interfaz `0.0.0.0` con un tiempo de espera de hasta 300 segundos para absorber la latencia de arranque inicial (*cold start*).
3. **Política de asignación de instancias y control presupuestario:** Con el propósito de optimizar el consumo de créditos y acotar los costos operativos dentro del marco del Trabajo Final de Grado, se configuró una política de instancias delimitada entre cero y una réplica (`-min-instances 0`, `-max-instances 1`). El escalado a cero libera los recursos cuando no existen llamadas activas.

El motor local ILSA y la extensión de navegador se enlazaron con las URLs públicas HTTPS de ambas instancias en Cloud Run. Esta integración confirmó la correcta transmisión de cargas textuales y la recuperación de activos para la sintaxis del avatar en tiempo pseudo-real, manteniendo liberados los núcleos de cómputo y la memoria del dispositivo cliente.

Capítulo 7

Pruebas, Resultados y Discusión

7.1. Corpus y metodología de adquisición de datos de LSA

El desarrollo del módulo de reconocimiento comenzó con un conjunto de prueba de la Lengua de Señas Americana (ASL). Ese material sirvió para poner a punto el pipeline inicial de extracción y de clasificación, antes de contar con grabaciones propias. La línea se descartó en las primeras iteraciones por dos motivos. El primero es lingüístico: la ASL no es intercambiable con la LSA, porque difiere en el léxico, en la gramática y, en varios ítems, en la forma de la seña. El segundo es de calidad: el conjunto presentaba etiquetas inconsistentes y videos espejados. El antecedente local LSA64, por su parte, emplea un protocolo de captura controlada con guantes de colores, una condición que no se reproduce en una videollamada con cámara web doméstica. Por esas razones, el entrenamiento usa un corpus propio de Lengua de Señas Argentina (LSA).

7.1.1. Definición y validación del vocabulario de 100 clases

El vocabulario objetivo abarca un conjunto de 100 señas delimitadas con el asesoramiento técnico de un informante calificado en LSA. La selección léxica priorizó dos dominios funcionales: identificación de personas y verbos de emergencia, por un lado, y asistencia al viajero (indicadores temporales, lugares, parentesco y saludos), por el otro.

La incorporación de las clases al modelo operó de manera condicionada al alcance de un piso mínimo de muestras en el dataset. En julio de 2026, con un umbral de 50 videos por clase, el modelo activo clasificó 91 señas. En agosto de 2026, el incremento del piso a 60 videos permitió incorporar las señas `chau`, `tener` y `años`. El conjunto llegó así a 94 clases. En septiembre de 2026, el conjunto se consolidó en 97 clases mediante la adición de `pasado`, `futuro` y `mio`. En octubre de 2026 el vocabulario se cerró en 100 clases con `gracias`, `ayudar` y `policia`, y completando todas las señas con 100 muestras.

Las 100 glosas se categorizan en: partículas interrogativas, afirmación y negación, dígitos, alfabeto dactilológico, pronombres, saludos, unidades temporales, relaciones de parentesco, lugares y un conjunto cerrado de verbos.

7.1.2. Protocolo de grabación y curación del dataset propio

La captura de video se ejecutó en resolución 1920×1080 píxeles con relación de aspecto 16:9. A fecha de septiembre de 2026, el dataset contabilizó 6 748 archivos en formato MP4

(promedio de 69 clips por seña) y 6 680 tensores en formato `.npy` con dimensiones $(16, 225)$ guardados en el directorio `dataset_landmarks_16frames`.

Para evitar desbalances, la selección de muestras limitó el cupo a un máximo de 60 clips por clase mediante aleatorización con semilla fija (42). La partición de datos reservó el 80 % de las secuencias para entrenamiento y el 20 % para validación, es decir, 48 y 12 clips por clase respectivamente. En el checkpoint del 15 de septiembre de 2026, este esquema totalizó 4 656 secuencias de entrenamiento y 1 164 de validación. Con el vocabulario de 100 clases, el mismo esquema produjo 4 800 secuencias de entrenamiento y 1 200 de validación (reentrenamiento del 15 de octubre de 2026).

Dado que la partición operó por clip y no por sujeto, secuencias registradas en una misma sesión de grabación se distribuyeron entre ambos conjuntos. Este factor explica por qué la exactitud observada en la fase de validación fuera de línea resultó superior a las mediciones obtenidas posteriormente frente a la cámara en tiempo real.

La aumentación de datos se aplicó exclusivamente durante la fase de entrenamiento mediante transformaciones al vuelo:

- Escala isotrópica aleatoria en el rango de 0,85 a 1,15.
- Ruido gaussiano sobre las coordenadas, con desvío de 0,012 en los puntos de pose y de 0,027 en los de las manos.
- Rotación tridimensional (ángulos de guiñada o *yaw* $\pm 15°$, cabeceo o *pitch* y balanceo o *roll* $\pm 8°$).
- Recorte temporal aleatorio de hasta un 15 % en los extremos de la secuencia.
- Deformación de velocidad de articulación en el rango de 0,80 a 1,25.
- Anulación aleatoria de hasta 3 fotogramas consecutivos con posterior interpolación lineal.

Un multiplicador virtual de factor 25 generó variaciones estocásticas en cada época de entrenamiento. Se omitió la inversión horizontal (*flipping*) para preservar la lateralidad nativa de las señas asimétricas. El conjunto de validación no recibió aumentaciones ni repeticiones sintéticas.

7.2. Evaluación empírica del clasificador gestual (TinySkeletonClassifier)

El criterio de selección del modelo integrado en el producto final no respondió a la exactitud de validación fuera de línea, sino al porcentaje de aciertos en la evaluación frente a la cámara local en vivo (modos Top-1 y Top-3, una toma por seña, mano diestra y modo automático). A lo largo del desarrollo, esta métrica orientó el descarte de redes de mayor complejidad, búsquedas automatizadas de hiperparámetros y líneas de aprendizaje por transferencia, y ratificó la elección del modelo base de 16 fotogramas.

7.2.1. Protocolo de evaluación en cámara real

La prueba de integración se ejecutó en la estación local mediante un script interactivo que solicitó la articulación consecutiva de las señas del vocabulario. El clasificador emitió

la glosa predicha junto con su nivel de confianza. Esta evaluación enfrentó al modelo a variaciones de iluminación, óptica y posicionamiento no presentes en el conjunto de entrenamiento.

7.2.2. Comparativa entre métricas fuera de línea y rendimiento en cámara

La Tabla 7.1 presenta la brecha sistemática entre la exactitud obtenida en validación estática y el rendimiento registrado en tiempo real.

Cuadro 7.1: Exactitud de validación fuera de línea frente al acierto en cámara real (Top-1 y Top-3 crudos, sin colapsar homógrafos).

Checkpoint	**Fecha**	**Val. Top-1**	**Cámara Top-1**	**Cámara Top-3**
16 fotogramas, 91 clases	29–30/07	97,36 %	61,5 %	83,5 %
32 fotogramas, 128/4/2	31/07	96,48 %	71,4 %	87,9 %
32 fotogramas, 64/2/1	31/07	97,36 %	67,0 %	90,1 %
Optuna v1, 32 fotogramas	31/07	97,69 %	66,0 %	84,6 %
Baseline 16f, luz ambiental, 94 clases	18/08	98,40 %	67,0 %	89,4 %
Baseline 16f, iluminación dirigida, 94 clases	28/08	98,40 %	86,2 %	95,7 %
Baseline 16f, 97 clases	15–17/09	98,28 %	77,3 %	94,8 %

Tres factores explican esta diferencia:

1. **Disparidad de dominio:** La validación reutilizó encuadres de los videos originales, mientras que la cámara web del cliente introdujo ópticas y distorsiones distintas.
2. **Sensibilidad a la iluminación:** La misma arquitectura de 16 fotogramas incrementó su rendimiento en cámara del 67,0 % al 86,2 % Top-1 al controlar la iluminación del entorno (sesión del 28 de agosto), sin modificar los pesos de la red.
3. **Varianza por toma única:** Evaluaciones repetidas sobre un conjunto de 36 caracteres evidenciaron fluctuaciones de entre 11 y 19 puntos porcentuales asociadas a la variabilidad de articulación en una sola toma.

En paralelo, se evaluó una línea de aprendizaje por transferencia (*Transfer Learning*) sobre coordenadas articulares que alcanzó exactitudes de 0,98 y 0,99 en reportes estáticos sobre 1 084 muestras. No obstante, esos reportes incluían clases sin soporte y no completaron el protocolo de evaluación en cámara, por lo que la aproximación se descartó a favor del modelo *TinySkeleton* entrenado desde cero.

7.2.3. Impacto de la resolución temporal: 16 frente a 32 fotogramas

En julio de 2026, con un vocabulario de 91 clases y 50 videos por clase, se evaluó la hipótesis de que un mayor contexto temporal (32 fotogramas) mejoraría la discriminación de gestos dinámicos extensos. En la Tabla 7.2 y las Figuras 7.1 y 7.2 se detallan los resultados obtenidos.

Cuadro 7.2: Evaluación de la resolución temporal en julio de 2026 (91 clases, 50 videos por clase).

Iteración	**Arquitectura**	**Val. Top-1**	**Cámara Top-1**	**Cámara Top-3**
A, 16 fotogramas	128 dim / 4 cabezas / 2 capas	97,36 %	61,5 %	83,5 %
B, 32 fotogramas	128 dim / 4 cabezas / 2 capas	96,48 %	71,4 %	87,9 %
C, 32 fotogramas	64 dim / 2 cabezas / 1 capa	97,36 %	67,0 %	90,1 %

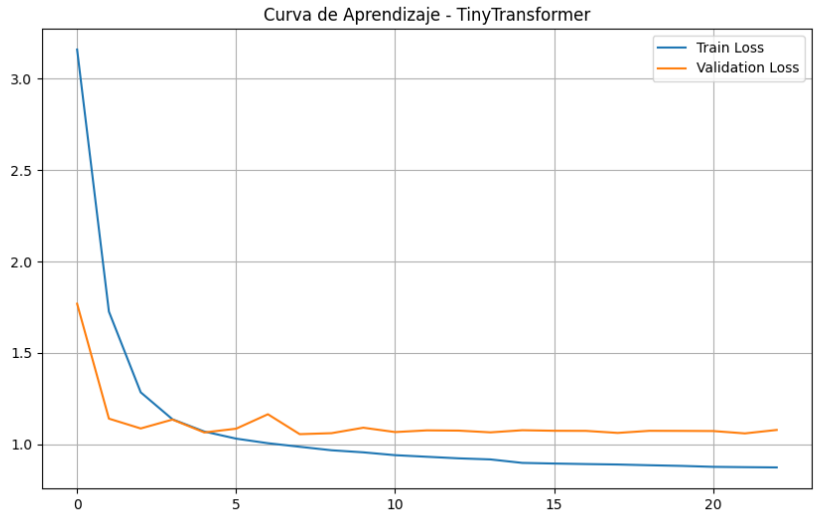


Figura 7.1: Curva de aprendizaje para la iteración A (16 fotogramas, 29/07/2026). La pérdida de validación se estabiliza cerca de 1,05 desde la época 7, mientras la pérdida de entrenamiento sigue en descenso.

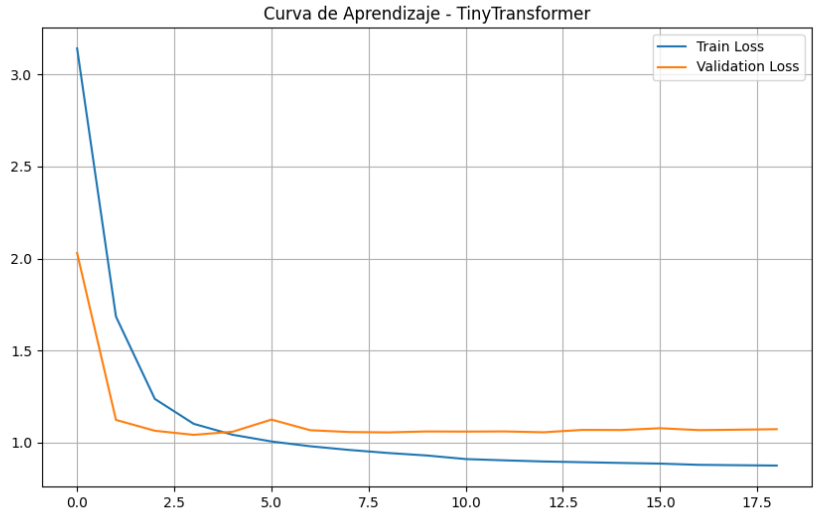


Figura 7.2: Curva de aprendizaje para la iteración B (32 fotogramas, 128 dimensiones, 4 cabezas y 2 capas, 31/07/2026). La pérdida de validación alcanza la estabilidad en 1,04 desde la época 4.

En las pruebas iniciales de julio, la ventana de 32 fotogramas aportó una ganancia de 9,9 puntos en el Top-1 frente a la cámara (71,4 % frente a 61,5 %). Sin embargo, a partir de agosto, con la expansión del dataset a 60 muestras por clase y la incorporación de aumentaciones avanzadas, la comparación favoreció nuevamente a la ventana de 16 fotogramas. La arquitectura de 16 fotogramas ofreció una mayor capacidad de generalización sobre señas estáticas y redujo la redundancia temporal de datos. Por eso quedó como la configuración definitiva.

7.2.4. Búsqueda automatizada con Optuna y análisis de sobreajuste

La optimización de hiperparámetros mediante Optuna empleó una validación cruzada de 3 pliegues sobre el conjunto de entrenamiento con poda de ensayos no prometedores. El objetivo del estudio fue minimizar la pérdida de validación.

Ronda de julio de 2026. Sobre secuencias de 32 fotogramas y 40 ensayos, la búsqueda seleccionó 128 dimensiones, 8 cabezas de atención, 2 capas, dropout 0,30, tamaño de lote de 16 y tasa de aprendizaje $3{,}77 \times 10^{-5}$. Tras el reentrenamiento, la pérdida de validación disminuyó a 0,087 y la exactitud fuera de línea alcanzó el 97,69 %. En las pruebas en vivo, la exactitud registró un 66,0 % Top-1 y un 84,6 % Top-3, valores inferiores a los de la iteración B configurada manualmente.

Ronda de agosto de 2026. El estudio `tinyskeleton_lsa_v2` (40 ensayos) incluyó la longitud temporal dentro del espacio de búsqueda (candidatos de 8, 12 y 16 fotogramas). La mejor prueba seleccionó 12 fotogramas, 256 dimensiones, 2 cabezas, 2 capas y dropout 0,57, con una pérdida media de validación de 0,137. En cámara, dicho modelo obtuvo un 80,9 % Top-1 frente al 86,2 % del modelo base de 16 fotogramas en las mismas

condiciones. El modelo base de 16 fotogramas superó a las variantes optimizadas por Optuna principalmente en la precisión sobre el alfabeto dactilológico.

El sobreajuste observado en las curvas iniciales (Figuras 7.1 y 7.2) se atenuó en las versiones de septiembre y octubre mediante el incremento del volumen de datos y la aplicación de parada temprana (*early stopping*). En las Figuras 7.3 y 7.5 se aprecia cómo la brecha entre la pérdida de entrenamiento y la de validación se redujo sustancialmente.

7.2.5. Diagnóstico de confusiones mediante feature engineering

Para diagnosticar la proximidad geométrica entre clases sin alterar la arquitectura neuronal, se implementó un descriptor auxiliar de 26 mediciones extraídas directamente del tensor de 225 coordenadas:

- **Relaciones mano-rostro (6 features):** Distancias normalizadas por el ancho de los hombros para identificar puntos de articulación.
- **Trayectoria de muñeca (6 features):** Desviación estándar y rango dinámico en los ejes x, y y z para distinguir gestos estáticos de dinámicos.
- **Configuración digital (14 features):** Extensión, curvatura y separación entre puntas de dedos vecinos, normalizadas por la distancia entre la muñeca y el nudillo del dedo medio.

El grado de separabilidad se calculó mediante el estadístico del área bajo la curva (AUC) de Mann-Whitney, definido como $\text{sep} = |\text{AUC} - 0{,}5| \times 2$ (donde 1 indica distribuciones disjuntas y 0 solapamiento total). Como control suplementario, un clasificador 1-NN *leave-one-out* sobre los tensores aplanados alcanzó un 88,4 %, un valor optimista porque los clips de una misma sesión se parecen entre sí.

Con las 12 medidas iniciales, de lugar y de movimiento, los pares del alfabeto y de los dígitos quedaban entre 0,21 y 0,30. La familia de configuración de dedos los llevó a valores entre 0,97 y 1,00:

- `2` frente a `5`: Separados por la distancia entre el dedo medio y anular (sep = 1,00).
- `D` frente a `O`: Separados por la curvatura del dedo índice (sep = 0,97).
- `M` frente a `N`: Separados por la flexión del meñique (sep = 1,00).

Por el contrario, el par `0` (número) y `O` (letra) presentó una separabilidad de 0,53, lo que confirma su condición de homógrafos. El cruce con la sesión de cámara del 17/09 atribuyó 17 de los 22 fallos a la ejecución o a la captura. Dos correspondieron al par `V`/`viernes`, ambiguo en el dato, y tres fueron casos límite: `O`/`0`, `años`/`G` y `ojo`/`vivir`.

7.2.6. Impacto cuantitativo de la corrección de relación de aspecto

El efecto de la corrección de la relación de aspecto (conversión de capturas 4 : 3 al marco estandarizado 16 : 9 con bandas de relleno) se evaluó en primera instancia fuera de línea sobre 36 tomas de letras y dígitos, con el modelo del 29 de agosto. La Tabla 7.3 resume la ganancia en exactitud obtenida mediante la transformación analítica de la coordenada y ($\times$1,333).

Cuadro 7.3: Efecto de la corrección de relación de aspecto sobre 36 tomas de letras y dígitos (checkpoint del 29/08).

Configuración de entrada	**Exactitud Top-1**
Captura original en relación 4:3	66,7 %
Corrección analítica en eje y ($\times$1,333)	88,9 %
Corrección simulada en ejes y y z	75,0 %

La corrección en el eje y aportó un incremento de 22,2 puntos porcentuales en la exactitud. Un caso relevante ocurrió con la letra `I`, la cual se clasificaba erróneamente como la seña `ojo` con confianza de 1,000 bajo la proporción 4:3, e ingresó a la clase correcta `I` con probabilidad 1,000 tras aplicar el escalado analítico. La corrección simultánea sobre el eje z redujo el desempeño, lo que indica que el relleno de márgenes no debe distorsionar la componente de profundidad relativa.

7.2.7. Desempeño final del clasificador en tiempo real

La configuración retenida es el baseline de 16 fotogramas, 128 dimensiones, 4 cabezas de atención, 2 capas Transformer y dropout 0,4. Esa arquitectura no cambió desde septiembre. Cambiaron el extractor y el vocabulario.

La última evaluación en cámara corresponde al checkpoint del 15 de septiembre de 2026, de 97 clases, entrenado durante 35 épocas con el extractor de Python. Ese checkpoint alcanzó una exactitud de validación del 98,28 % y una pérdida de 0,062. En la sesión del 17 de septiembre (una toma por seña, MediaPipe 0.10.21 y relleno 16:9), acertó 75 de las 97 señas en Top-1 (77,3 %) y 92 de 97 en Top-3 (94,8 %). Las señas que quedaron fuera del Top-3 fueron `donde`, `ellos`, `familia`, `nosotros` y `repetir`. El subconjunto de 36 letras y dígitos alcanzó un 88,9 % en Top-1 y un 100 % en Top-3. Con el homógrafo `O`$\equiv$`0` como acierto, el Top-1 sobre las 97 clases sube al 78,4 %. La confianza media fue de 0,971 en los aciertos y de 0,833 en los errores: la red se equivoca con certeza alta, y el umbral de confianza no alcanza, por sí solo, para filtrar esos fallos.

El reentrenamiento del 21 de septiembre de 2026 conservó la arquitectura y las 97 clases, y reemplazó el extractor de Python por Holistic en navegador, el mismo que usa la extensión. Alcanzó una pérdida de validación de 0,052 y una exactitud de 99,14 %. Las Figuras 7.3 y 7.4 corresponden a esa corrida.

El modelo integrado en el prototipo surge del reentrenamiento del 15 de octubre de 2026, que extendió el vocabulario a 100 clases. Con 4 800 secuencias de entrenamiento y 1 200 de validación, la parada temprana detuvo el ajuste en la época 34 y retuvo el checkpoint de la época 19, el de menor pérdida de validación (0,0528), con una exactitud de 99,08 %. La Figura 7.5 muestra su curva de aprendizaje. La red tiene 414 309 parámetros entrenables, cerca de 1,66 MB en precisión simple. Un reentrenamiento posterior, del 20 de octubre, elevó el cupo a 100 clips por clase (8 000 y 2 000 secuencias) y obtuvo una pérdida de 0,028 y una exactitud de 99,0 %, pero no se integró al prototipo. Ninguno de los dos checkpoints de 100 clases cuenta con una evaluación en cámara archivada. Las cifras en vivo de esta sección describen, por lo tanto, el modelo de 97 clases. Las tres señas nuevas y el cambio de extractor no tienen todavía una medición frente a la cámara.

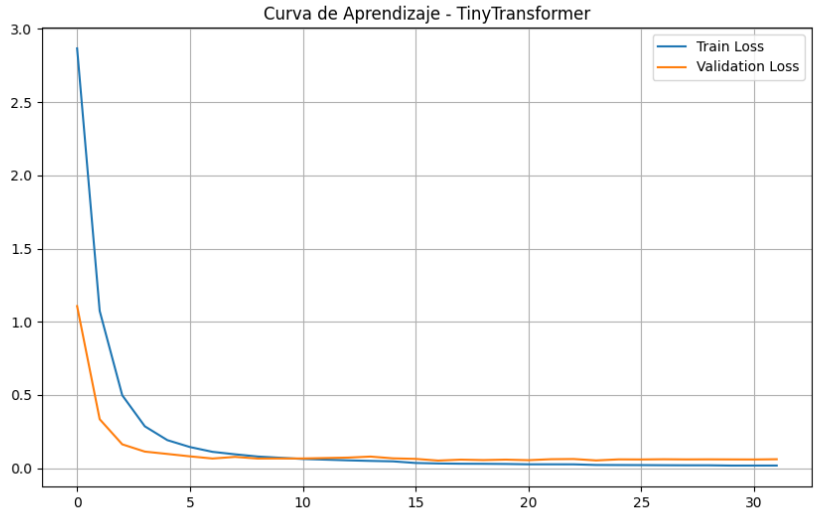


Figura 7.3: Curva de aprendizaje del reentrenamiento del 21/09/2026 (16 fotogramas, 97 clases). La pérdida de validación mínima fue de 0,052 (exactitud Top-1 fuera de línea del 99,14 %).

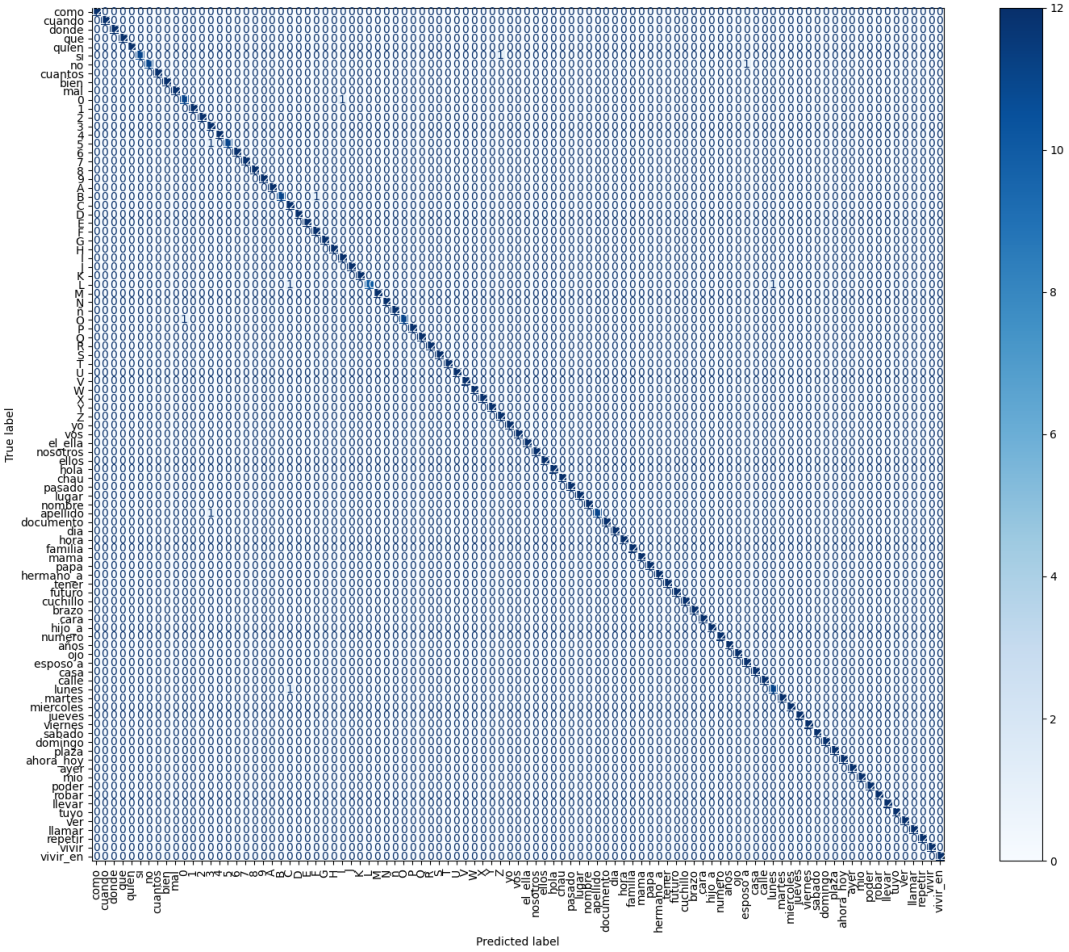


Figura 7.4: Matriz de confusión en validación fuera de línea para la corrida del 21/09/2026 (97 clases). La concentración de masa en la diagonal refleja la alta separabilidad del dataset estático.

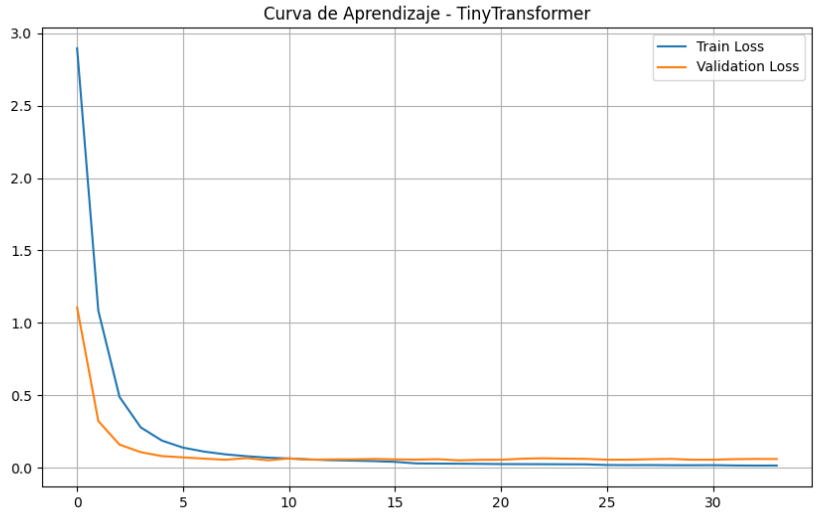


Figura 7.5: Curva de aprendizaje del reentrenamiento del 15/10/2026 (16 fotogramas, 100 clases), checkpoint integrado en el prototipo. La pérdida de validación mínima, 0,0528, se alcanza en la época 19 (exactitud Top-1 fuera de línea del 99,08 %).

7.3. Evaluación de los módulos de traducción semántica

La construcción de los corpus de entrenamiento y el ajuste de los modelos de traducción se desarrollaron mediante una metodología iterativa orientada al descubrimiento de casos límite de cobertura lingüística. Debido a la inexistencia de corpus públicos paralelos de español y glosas adaptados al dominio de aplicación y a las restricciones sintácticas del prototipo, se diseñó un pipeline de sintaxis asistida por modelos de lenguaje de gran escala (LLM).

7.3.1. Estrategia de generación asistida del dataset

Para garantizar la precisión semántica y la alineación lógica del corpus de entrenamiento, se empleó Gemini a partir de un diseño riguroso de prompts estructurados. La generación de pares sintéticos se fundamentó en tres restricciones operativas primarias:

1. **Respeto de la gramática de glosas:** Aplicación estricta de las reglas de orden sintáctico especificadas para el prototipo (Sección 3.1.2).

2. **Cierre léxico estricto:** Restricción del vocabulario al conjunto cerrado de signos soportados por el pipeline de traducción directa.

3. **Cobertura de fenómenos expresivos:** Inclusión sistemática de estructuras complejas como modalidades interrogativas y negativas, marcas temporales, deletreo nominal y expresiones numéricas.

El prompt solicitó explícitamente múltiples variantes pragmáticas de oraciones en español para cada secuencia de glosas válida. Cada borrador generado por el modelo

atravesó una fase de revisión, corrección y validación manual antes de su incorporación definitiva al corpus.El ciclo de entrenamiento adoptó un enfoque activo guiado por errores. Tras la conclusión de un entrenamiento, el modelo actualizado se evaluó sobre un conjunto de prueba restringido (entre 10 y 20 oraciones) orientado a explorar variaciones verbales, sintácticas y pragmáticas no observadas. Las fallas y huecos de cobertura identificados en esta instancia se utilizaron como señal para solicitar a Gemini nuevas variaciones estructuradas bajo el mismo control de calidad, las cuales enriquecieron las versiones subsiguientes del corpus.

7.3.2. Corrida histórica previa a la compresión de tokens (Canal Directo)

La primera corrida conservada en los registros del proyecto corresponde al 31 de agosto de 2026. Esta prueba evaluó únicamente el canal directo (Lengua de Señas Argentina a Español), dado que el módulo inverso para oyentes aún no se encontraba desarrollado. Las secuencias de glosas empleadas pertenecen a una etapa previa a la optimización de tokenización que agrupa caracteres de deletreo y dígitos numéricos consecutivos en un token unificado.La Tabla 7.4 sintetiza las métricas obtenidas por las cinco familias de modelos evaluadas en esa etapa inicial sobre un conjunto de prueba de 115 ejemplos.

Cuadro 7.4: Canal directo, evaluación histórica antes de la compresión de tokens de deletreo y dígitos. Muestra de 115 oraciones. Índices en escala 0–100.

Modelo	**Exacto**	**BLEU**	**ROUGE-L**	**METEOR**	**ms**
Qwen2.5 0,5B	86,09	88,60	92,52	92,74	309,09
Qwen2.5 1,5B	87,83	87,77	93,17	92,42	349,94
Qwen2.5 3B	93,04	95,22	97,06	95,79	551,64
Llama 3.2 1B	88,70	88,35	95,56	94,61	247,38
SmolLM2 1,7B	87,83	92,35	94,53	93,32	340,46

7.3.3. Resultados finales y reevaluación semántica

El dataset consolidado alcanzó una dimensión final de 791 pares para el canal directo y 676 para el canal inverso. Las pruebas finales del 25 de octubre de 2026 utilizaron una partición fija del 15 % del corpus para la evaluación (semilla aleatoria 3407), correspondiente a 119 oraciones en el canal directo y 102 oraciones en el canal inverso. Esta corrida reevaluó los adaptadores previamente entrenados con el objetivo de incorporar métricas semánticas profundas basadas en embeddings, específicamente `bertscore_f1` (BERTScore en español) y `comet_score` (`Unbabel/wmt22-comet-da`), en adición a los índices n-gramáticos tradicionales (Sección 3.3).La inferencia se ejecutó localmente sobre binarios cuantizados en formato GGUF (`Q4_K_M`) con temperatura 0, un límite de generación de 24 tokens para el canal directo y 48 tokens para el canal inverso. La métrica de acierto exacto considera aceptable cualquiera de las referencias semánticas equivalentes definidas en el dataset. La latencia reportada mide de forma exclusiva el tiempo de procesamiento en CPU local, sin incluir los tiempos de transporte de red.

Cuadro 7.5: Canal directo (LSA → Español), resultados finales (119 oraciones de evaluación). Índices en escala 0–100.

Modelo	Exacto	BLEU	ROUGE-L	METEOR	BERT	COMET	ms
Qwen2.5 0,5B	92,44	95,58	95,78	95,51	99,16	97,47	141,05
Qwen2.5 1,5B	94,96	96,32	98,02	97,35	99,20	97,36	256,30
Qwen2.5 3B	95,80	97,88	98,39	97,84	99,20	97,56	514,38
Llama 3.2 1B	89,92	92,64	95,94	96,13	98,58	96,83	231,28
SmolLM2 1,7B	90,76	93,53	96,48	95,97	98,85	97,12	438,31

Cuadro 7.6: Canal inverso (Español → LSA), resultados finales (102 oraciones de evaluación). Índices en escala 0–100.

Modelo	Exacto	BLEU	ROUGE-L	METEOR	BERT	COMET	ms
Qwen2.5 0,5B	87,25	91,29	96,20	91,25	98,86	94,28	155,97
Qwen2.5 1,5B	91,18	94,14	97,17	92,75	99,22	95,98	290,82
Qwen2.5 3B	89,22	92,51	96,85	92,71	99,21	95,45	537,58
Llama 3.2 1B	89,22	92,66	96,36	92,89	99,13	95,29	245,70
SmolLM2 1,7B	77,45	82,90	91,54	86,31	97,68	92,37	458,39

En el canal directo (Tabla 7.5), Qwen2.5 3B obtiene el desempeño óptimo en la totalidad de las métricas evaluadas, con un acierto exacto del 95,80 % (114 de 119 oraciones). Qwen2.5 1,5B se ubica a un único acierto de distancia (94,96 %) y registra un empate en el valor de BERTScore (99,20). La métrica COMET exhibe un comportamiento uniforme entre los modelos de la arquitectura Qwen, con variaciones contenidas entre 97,36 y 97,56.En el canal inverso (Tabla 7.6), la jerarquía de los modelos presenta una variación relevante. Qwen2.5 1,5B encabeza la evaluación con un acierto exacto del 91,18 % (93 de 102 oraciones) y supera a la versión de 3B (89,22 %). Llama 3.2 1B iguala el nivel de acierto exacto del modelo Qwen2.5 3B y alcanza una ligera ventaja en la métrica METEOR (92,89). SmolLM2 1,7B registra los valores más bajos en ambas direcciones, con una caída pronunciada en el canal inverso (77,45 %).

7.3.4. Análisis cualitativo de errores en la inferencia

Un examen detallado sobre las discrepancias producidas por el modelo desplegado en producción (Qwen2.5 1,5B) revela la naturaleza de las desviaciones:

- **Canal inverso (9 errores en 102 oraciones):** Dos casos corresponden a permutaciones de sinonimia válida en el orden de las glosas (por ejemplo, la entrada `Mi cara` genera `CARA MIO` frente a la referencia `ROSTRO MIO`). Tres casos involucran la adición o supresión de marcas temporales contextuales (como la adición de `PASADO` ante la consulta `¿Cuándo fuiste a la plaza?`). Los tres casos restantes representan fallas de sustitución léxica directa, donde la métrica COMET registra caídas puntuales hacia valores de 74,6, 65,0 y 72,6, sensiblemente inferiores al promedio global de 95,98.
- **Canal directo (6 errores en 119 oraciones):** Las desviaciones responden a errores ortográficos menores en la reconstrucción del texto en español (por ejemplo, `Repeten` en lugar de `Repiten`), variaciones no especificadas en la concordancia de

género o la omisión de pronombres oblicuos implícitos. Asimismo, se observan casos donde la salida rescata correctamente elementos presentes en la glosa de entrada pero ausentes en la referencia simplificada, situación que el índice de acierto exacto penaliza de manera estricta.

7.3.5. Análisis de asimetría direccional

La comparación de los resultados refleja un comportamiento asimétrico inherente a la naturaleza lingüística de ambos canales:

- **Canal directo (LSA → Español):** Representa una tarea de *expansión sintáctica y morfológica*. A partir de secuencias telegráficas de glosas carentes de flexión verbal, artículos y conectores, el modelo reconstruye la fluidez sintáctica de la lengua natural.
- **Canal inverso (Español → LSA):** Constituye una tarea de *compresión y reordenamiento estructurado*. El modelo debe abstraer la estructura del español y mapear los elementos con arreglo a las restricciones sintácticas rígidas de la secuencia manual.

La asimetría se constata empíricamente en la diferencia de acierto exacto entre ambos canales para un mismo modelo. La brecha a favor del canal directo asciende a 3,78 puntos porcentuales en Qwen2.5 1,5B, 6,58 puntos en Qwen2.5 3B y hasta 13,31 puntos en SmolLM2 1,7B. Esta especialización se ratificó durante las pruebas de despliegue: el intercambio del adaptador binario del canal directo hacia el endpoint del canal inverso provocó la generación de respuestas directas en español ante solicitudes dirigidas al módulo de sintaxis para personas sordoimpertinentes, lo que demuestra la fijación de la dirección de traducción dentro de los pesos del adaptador LoRA.

7.3.6. Compromiso entre latencia y capacidad lingüística

El tiempo de generación del binario cuantizado escala en relación directa con el número de parámetros de la arquitectura. Dentro de la familia Qwen2.5, la latencia media se incrementa aproximadamente en un 100 % al pasar del bloque de 1,5B (256,30,ms en canal directo) al bloque de 3B (514,38,ms en canal directo). Dado que la ganancia en exactitud sintáctica del modelo de 3B resulta marginal en el canal directo y desfavorable en el canal inverso, la variante Qwen2.5 1,5B se consolida como la arquitectura óptima para el despliegue en el servidor principal, con Llama 3.2 1B como alternativa de respaldo.El servicio desplegado sobre infraestructura Google Cloud Run (configuración de 4 vCPU y 8,GiB de RAM) incorporó un mecanismo de preservación de caché de claves y valores (KV cache) para el prefijo de contexto del prompt. Esta estrategia evita el costo computacional de la reevaluación completa del prompt en CPU, proceso que demanda aproximadamente 8 segundos en instancias frías o ante discrepancias de prefijo.

7.4. Evaluación del sistema integrado y rendimiento en tiempo real

7.4.1. Validación funcional de extremo a extremo

La integración del prototipo avanzó en cinco hitos funcionales. Desde el segundo, cada uno se verificó de punta a punta en una videollamada de Google Meet:

1. **Aplicación de escritorio:** La captura y la clasificación de 91 señas corrían en una aplicación con OpenCV, fuera de la videollamada.

2. **Extensión web y motor local:** La extensión Manifest V3 y el motor ILSA cerraron el circuito en Google Meet. El traductor corría en la CPU del equipo, y la oración se mostraba como subtítulo.

3. **Servicio semántico remoto:** El motor clasificaba 97 señas, y la traducción pasó a un servidor semántico expuesto mediante un túnel cifrado (*ngrok*).

4. **Traductor en la nube y vocabulario de 100 señas:** El servicio semántico pasó a Google Cloud Run, y el motor ILSA incorporó el clasificador de 100 clases con extracción de Holistic en navegador.

5. **Canal oyente completo:** Se sumó el servicio de poses en Cloud Run. El canal oyente quedó con transcripción local mediante Vosk, estructuración remota en glosas y reproducción del avatar en la videollamada.

Para evaluar la lógica de control del motor local (`RepeatGate`) y el temporizador de cierre de enunciado, se ejecutó una batería de pruebas resumida en la Tabla 7.7.

Cuadro 7.7: Resultados de la batería de pruebas sobre la lógica de control del motor local.

Caso de prueba	**Entrada ingresada**	**Resultado emitido**
Dígitos repetidos	`1 1 1 2 2`	Aceptación total: `1 1 1 2 2`
Letra repetida tres veces	`A A A B`	Filtrado a máximo de dos: `A A B`
Rebote del clasificador	`HOLA HOLA`	Descarte de duplicado inmediato: `HOLA`
Repetición no consecutiva	`HOLA VER HOLA`	Conservación de la secuencia completa
Glosa repetida descartada a los 3 s	Glosa repetida	El cierre se desplaza a los 7 s
Movimiento continuo hasta 5 s	Sin glosa nueva	Cierre de enunciado a los 9 s
Deletreo dactilológico	`J U A N`	Conversión directa a «Juan.» sin consulta a nube
Deletreo con ambigüedad	`T I N A`	Derivación al traductor por coincidencia con `OJO`

Durante la integración con la interfaz de Google Meet se corrigieron fallas técnicas de manipulación del flujo multimedia. La Tabla 7.8 detalla las anomalías detectadas y la intervención realizada.

Cuadro 7.8: Fallas detectadas en la integración con Google Meet y correcciones aplicadas.

Falla detectada	**Intervención de ingeniería aplicada**
Bloqueo del inicio de video en Meet por intercepción permanente de cámara.	Activación del reemplazo de la pista únicamente con el motor ILSA encendido.
Pérdida de enlace al recargar la extensión con la pestaña de la reunión abierta.	Después de recargar la extensión, se recarga manualmente la pestaña de Meet.
Detección de falsas manos provocada por movimiento del torso.	Exigencia de un número mínimo de cuadros consecutivos con mano detectada antes de abrir el gesto.
Interferencia en la captura por movimiento de personas en el fondo.	Delimitación del área de actividad basada exclusivamente en los puntos articulares de las manos.
Latencia por baja tasa de cuadros en umbrales estáticos.	Conversión de las reglas de espera de recuento de fotogramas a milisegundos reales.
Sobrecarga de CPU en la transferencia de imágenes hacia la ventana *offscreen*.	Reducción de la resolución de los cuadros transferidos a 240 píxeles de ancho con compresión ajustada.
El ejecutable no abría en equipos sin el entorno de Visual C++ ni encontraba las bibliotecas de PyTorch.	Inclusión de las bibliotecas dinámicas requeridas dentro de la carpeta de distribución del ejecutable.

7.4.2. Análisis de latencias totales por etapa del pipeline

El tiempo de respuesta percibido en el canal directo integra retardos de procesamiento numérico y pausas configuradas en la lógica del sistema. La Tabla 7.9 detalla la contribución de cada etapa.

Cuadro 7.9: Desglose de los componentes del retardo en el canal directo.

Etapa del pipeline	**Duración / Retardo**	**Naturaleza del parámetro**
Detección de fin de seña	$\approx 0{,}4\,\text{s}$	Configurado (12 fotogramas sin manos).
Inferencia del clasificador	0,78 ms (CPU) / 1,08 ms (GPU)	Medido el 31/07 sobre el modelo de 32 fotogramas y 91 clases.
Separación mínima entre señas	1,0 s	Configurado (tiempo de reposo inter-gestual).
Pausa de cierre de enunciado	4,0 s	Configurado (inactividad para disparar traducción).
Generación de oración (LLM)	256,30 ms	Medido el 25/10 sobre el GGUF de Qwen2.5 1,5B, canal directo. No representa el servicio en Cloud Run.
Reevaluación de prompt (CPU)	$\approx 8\,\text{s}$	Valor típico observado en CPU ante pérdida de caché KV.
Arranque en frío del servicio	1–3 min	Ocurre únicamente si la instancia escaló a cero.
Permanencia de subtítulo	8,0 s	Configurado (visibilidad en lienzo virtual).

El componente determinante de la espera en la comunicación es la pausa fija de 4,0 segundos, introducida deliberadamente para asegurar la conclusión del enunciado en LSA antes de consultar al modelo de lenguaje.

7.4.3. Consumo de recursos de hardware: máquina de desarrollo frente a equipo objetivo

En la Tabla 7.10 se detalla el comportamiento del sistema evaluado sobre las dos plataformas de hardware del proyecto durante la versión con traductor local (13 de septiembre de 2026).

Cuadro 7.10: Comparativa de desempeño del sistema según la plataforma de hardware (con traductor local).

Componente	**Estación de desarrollo (GPU)**	**Equipo objetivo (Laptop sin GPU)**
Extracción MediaPipe	Fluida dentro de Meet.	Pocos fotogramas por segundo; umbrales de captura reajustados en milisegundos.
Clasificador local	Menos de 2 ms por inferencia (medido).	Sin demora perceptible (observación cualitativa).
Traductor local	Respuesta fluida.	Espera perceptible, incompatible con el ritmo de una llamada.

Esta evaluación motivó la reestructuración de la arquitectura: el clasificador permaneció en el cliente de borde (cerca de 1,66 MB para sus 414 309 parámetros), mientras que el modelo de lenguaje se reubicó en un servicio remoto en Google Cloud Run.

7.5. Discusión técnica y lecciones de ingeniería aprendidas

El desarrollo y la evaluación experimental del prototipo permitieron extraer las siguientes lecciones de ingeniería de software y aprendizaje profundo:

1. **La evaluación fuera de línea no garantiza el rendimiento en entorno real:** Aunque la validación estática superó el 96 % de exactitud en todas las iteraciones, la precisión del clasificador disminuyó al enfrentarse a la variabilidad de la cámara web. Las decisiones de selección debieron guiarse por las pruebas en vivo.

2. **La evaluación semántica exige referencias múltiples:** La utilización de una única referencia de traducción penalizó paráfrasis válidas en español. La incorporación de referencias múltiples corrigió este artefacto de medición.

3. **La calidad del dataset fue el factor determinante:** Las mejoras principales en la traducción semántica no provinieron de un modelo de lenguaje más grande. Provinieron de leer `dataset_trial`, ver qué estructura faltaba y sumar esos pares al corpus.

4. **Prevención de errores silenciosos de configuración:** La compatibilidad de formas matriciales en entornos como PyTorch puede ocultar discrepancias en la cantidad de cabezas de atención entre la instanciación y el entrenamiento. Esa discrepancia altera las probabilidades sin emitir excepciones. Por este motivo, la configuración debe serializarse y validarse junto con el checkpoint.

5. **Gestión de la memoria conversacional:** La inclusión de turnos previos en el prompt distanció las entradas de la distribución de entrenamiento aprendida por los adaptadores LoRA. La traducción por enunciados independientes garantizó la estabilidad de las respuestas.

6. **Impacto de la arquitectura híbrida:** La delegación de la traducción semántica a la nube resolvió la carga de cómputo del traductor en el equipo cliente. A cambio, introdujo la dependencia de la latencia de red y el tiempo de arranque en frío del servicio.

7. **Trabajos y mediciones pendientes:** Se identificó la necesidad de completar mediciones futuras que incluyan la caracterización de latencias de extremo a extremo con percentiles a partir de los registros del motor local, la recolección de métricas de uso de CPU/RAM en cliente con la traducción remota y la ejecución de pruebas de usabilidad con usuarios sordos.

Capítulo 8

Conclusiones y trabajo futuro

8.1. Conclusiones

8.1.1. Cumplimiento de objetivos del proyecto

8.1.2. Aportes técnicos y de accesibilidad

8.1.3. Impacto social en el marco de la Ley 27.710

8.2. Trabajo futuro

8.2.1. Implementación física del clúster semántico escalable

8.2.2. Expansión del corpus multiseñante y mitigación de variabilidad

8.2.3. Incorporación de rasgos no manuales (expresiones faciales)

8.2.4. Generalización a nuevas plataformas de comunicación virtual

Bibliografía

[1] República Argentina. *Ley 27.710 — Lengua de Señas Argentina*. Boletín Oficial. 2023.

[2] M. I. Massone y R. A. Martínez. *Curso de Lengua de Señas Argentina: Parte 1 y 2*. Cultura Sorda, 2012. URL: http://www.cultura-sorda.eu.

[3] S. K. Liddell y R. E. Johnson. «American Sign Language: The Phonological Base». En: *Sign Language Studies* 64 (1989), págs. 195-277.

[4] M. I. Massone. «El número y el género en la Lengua de Señas Argentina». En: *Signo & Seña* 2 (1993), págs. 73-99.

[5] C. Lugaresi et al. «MediaPipe: A Framework for Building Perception Pipelines». En: *Third Workshop on Computer Vision for AR/VR at IEEE CVPR*. 2019.

[6] Google. *MediaPipe: A Framework for Building Perception Pipelines*. https://google.github.io/mediapipe/. 2020.

[7] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang y M. Grundmann. *BlazePose: On-device Real-time Body Pose Tracking*. 2020. arXiv: 2006.10204 [cs.CV].

[8] F. Zhang et al. *MediaPipe Hands: On-device Real-time Hand Tracking*. 2020. arXiv: 2006.10214 [cs.CV].

[9] A. Moryossef, M. Müller y R. Fahrni. *pose-format: Library for viewing, augmenting, and handling .pose files*. https://github.com/sign-language-processing/pose. 2021.

[10] A. Vaswani et al. «Attention Is All You Need». En: *arXiv preprint arXiv:1706.03762* (2017).

[11] E. J. Hu et al. «LoRA: Low-Rank Adaptation of Large Language Models». En: *Proceedings of the International Conference on Learning Representations*. 2022.

[12] G. Gerganov. *llama.cpp: Port of Facebook's LLaMA model in C/C++*. 2023. URL: https://github.com/ggerganov/llama.cpp.

[13] W3C. *Media Capture and Streams*. https://www.w3.org/TR/mediacapture-streams/. 2024.

[14] W3C. *Media Capture from DOM Elements*. https://www.w3.org/TR/mediacapture-fromelement/. 2024.

[15] Google Chrome Developers. *Overview of Manifest V3*. https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3. 2024.

[16] Google Chrome Developers. *chrome.offscreen*. https://developer.chrome.com/docs/extensions/reference/api/offscreen. 2024.

[17] Google Chrome Developers. ***sandbox*** *(Manifest V3)*. https://developer.chrome.com/docs/extensions/reference/manifest/sandbox. 2024.

[18] INJU. *Diccionario de Señas*. https://senas.inju.gob.ar/. 2024.

[19] Confederación Argentina de Sordos. *Señario de términos y expresiones en Lengua de Señas Argentina*. https://senario.cas.org.ar/. 2024.

[20] V. Barán. *LSApp: Uniendo dos mundos*. https://lsapp.com.ar/. 2018.

[21] Blessit. *Interpretación Lengua de Señas — Central de intérpretes*. https://www.blessit.net/interpretacion-lengua-de-senas. 2024.

[22] Ministerio de Justicia de la Nación. *Características de la comunicación por videollamadas — Línea 144*. https://www.argentina.gob.ar/caracteristicas-de-la-comunicacion-por-videollamadas. 2024.

[23] F. Ronchetti, F. Quiroga, C. Estrebou, L. Lanzarini y A. Rosete. «LSA64: An Argentinian Sign Language Dataset». En: *XXII Congreso Argentino de Ciencias de la Computación (CACIC)*. 2016, págs. 794-803.

[24] I. Mindlin. «Reconocimiento de Lengua de Señas con redes neuronales recurrentes». Tesis de grado. Universidad Nacional de La Plata, Facultad de Informática, 2021. URL: http://sedici.unlp.edu.ar/handle/10915/129853.

[25] SignAll Technologies. *About SignAll Technologies*. https://www.linkedin.com/company/signall-technologies/about/. 2026.

[26] D. Li, C. Rodriguez, X. Yu y H. Li. «Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison». En: *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*. 2020, págs. 1459-1469.

[27] Hand Talk. *Acessibilidade digital em línguas de sinais*. https://www.handtalk.me/br/. 2025.

[28] HETAH. *Avatar traductor a lengua de señas*. https://hetah.net/avatar. 2024.

[29] R. S. Pressman y B. R. Maxim. *Software Engineering: A Practitioner's Approach*. 8th. McGraw-Hill, 2014.

[30] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama. «Optuna: A Next-generation Hyperparameter Optimization Framework». En: (2019), págs. 2623-2631.

[31] Alibaba Cloud. *Qwen2.5: A Family of Large Language Models*. https://qwenlm.github.io/. 2024.

[32] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra y T. Wolf. *SmolLM2: When Small Goes Big*. https://huggingface.co/blog/smollm2. 2024.

[33] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle et al. «The Llama 3 Herd of Models». En: *arXiv preprint arXiv:2407.21783* (2024).

[34] K. Papineni, S. Roukos, T. Ward y W.-J. Zhu. «BLEU: a Method for Automatic Evaluation of Machine Translation». En: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. 2002, págs. 311-318.

[35] C.-Y. Lin. «ROUGE: A Package for Automatic Evaluation of Summaries». En: *Text Summarization Branches Out*. 2004, págs. 74-81.

[36] S. Banerjee y A. Lavie. «METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments». En: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. 2005, págs. 65-72.

[37] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger e Y. Artzi. «BERTScore: Evaluating Text Generation with BERT». En: *International Conference on Learning Representations*. 2020.

[38] R. Rei, C. Stewart, A. C. Farinha y A. Lavie. «COMET: A Neural Framework for MT Evaluation». En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 2020, págs. 2685-2702.

Apéndice A

Vocabulario del clasificador

El clasificador distingue 100 clases. Cada clase cuenta con 100 clips. Miriam Rolls ayudó con la validación del conjunto. El cuadro A.1 separa el alfabeto, los dígitos y el léxico en grupos chicos, y cuenta cuántas clases hay en cada uno. La lista usa la lectura de la etiqueta: no es una traducción al español ni una definición de la seña.

Cuadro A.1: Señas del clasificador por categoría.

Categoría	**Cantidad**	**Señas**
Alfabeto manual	27	A–Z
Dígitos	10	0–9
Interrogativos	6	cómo, cuándo, dónde, qué, quién, cuántos
Afirmación y negación	2	sí, no
Valoración	2	bien, mal
Pronombres	5	yo, vos, él o ella, nosotros, ellos
Posesivos	2	mío, tuyo
Saludos y cortesía	3	hola, chau, gracias
Tiempo	6	pasado, futuro, día, hora, ahora u hoy, ayer
Días de la semana	7	lunes, martes, miércoles, jueves, viernes, sábado, domingo
Lugares	4	lugar, casa, calle, plaza
Identificación	5	nombre, apellido, documento, número, años
Parentesco	6	familia, mamá, papá, hermano o hermana, hijo o hija, esposo o esposa
Partes del cuerpo	3	brazo, cara, ojo
Objeto	1	cuchillo
Verbos	8	tener, poder, vivir, vivir en, ver, llamar, llevar, repetir
Auxilio	3	ayudar, robar, policía
Total	**100**	

Apéndice C

Arquitectura escalable de producción en la nube

La Figura C.1 pertenece a la Sección 5.6. La hoja va en horizontal para que el ingreso, el traductor y el servicio de archivos queden en una sola vista. El diagrama describe la proyección para varias videollamadas a la vez.

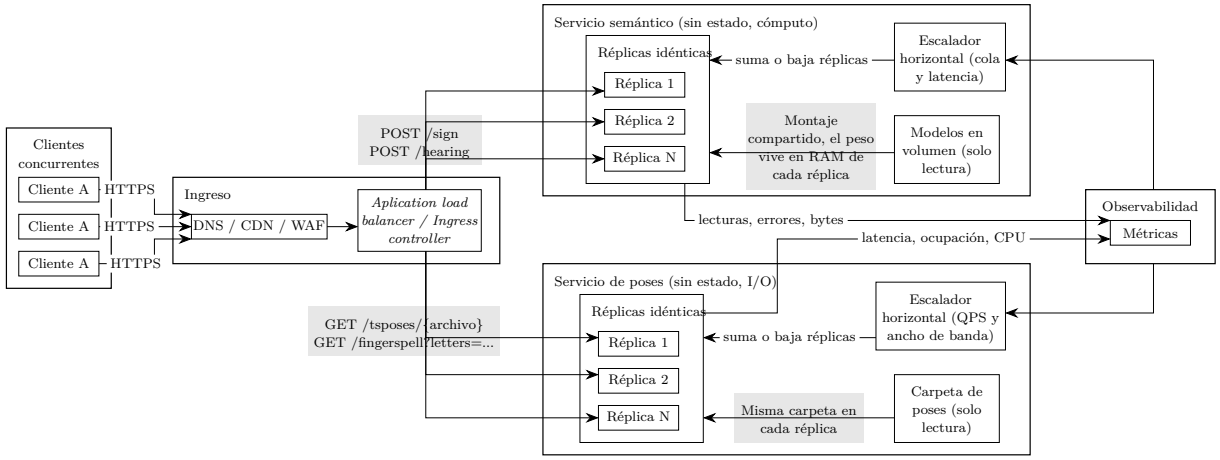


Figura C.1: Diagrama de la arquitectura del servidor, balanceo de carga y escalabilidad de servicios sin estado.